

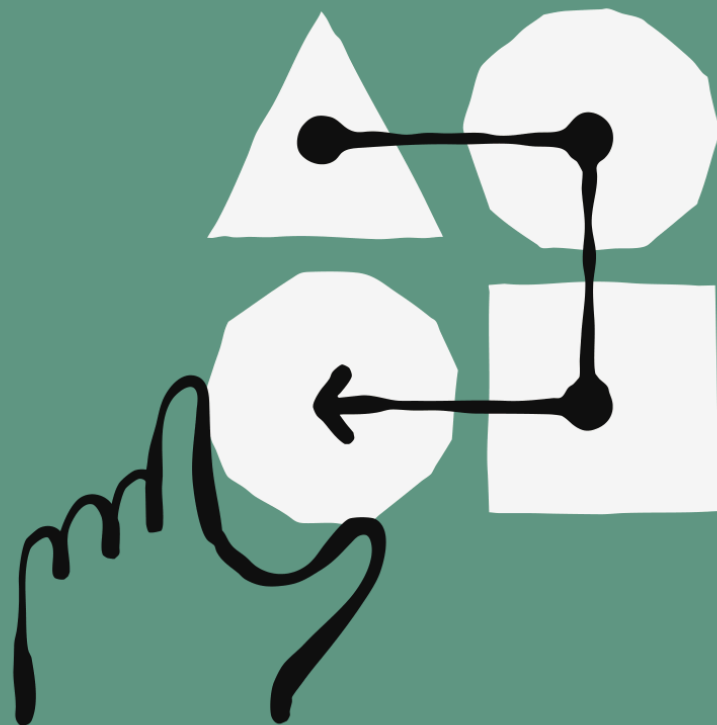
The Founder's Playbook: Building an AI-Native Startup



Contents

英文原文在上，中文直译在下。适合手机顺读、边读边查词、跟读、做笔记。

The startup lifecycle, rebooted for 2026	03
2026 年重启的初创公司生命周期	
What it means to be a founder is changing	05
创始人的含义正在改变	
Idea Stage	08
构思阶段	
MVP stage	15
MVP 阶段	
Launch stage	21
发布阶段	
Scale stage	25
规模化阶段	
Same job, new rules	31
同样的工作，新的规则	
Resources	33
资源	



Chapter 1

The startup lifecycle, rebooted for 2026

3

2026 年重启的初创公司生命周期

The startup lifecycle, rebooted for 2026

Chapter 1 The startup lifecycle, rebooted for 2026 AI is reshaping how startups are built. Founders who've never written a line of code are shipping production applications today, and the lean 10-person unicorn has gone from scrappy underdog story to deliberate plan of action.

第一章 为 2026 年重启的初创公司生命周期 AI 正在重塑初创公司的构建方式。从未写过一行代码的创始人如今正在交付生产级应用，而精简的 10 人独角兽公司已从拼搏的逆袭故事转变为深思熟虑的行动方案。

In 2026, AI can write production code, conduct market research, synthesize competitive landscapes, draft investor materials, and automate operational workflows. By eradicating the once-steep learning curves that even experienced technical founders faced in integrating the tools, platforms, and systems needed to bring their idea to life, AI has above all leveled the playing field around who can launch a startup or build a product.

2026 年，AI 能够编写生产级代码、开展市场调研、综合竞争格局、起草投资者材料，并自动化运营工作流。通过消除曾让经验丰富的技术创始人在整合将想法变为现实所需的工具、平台和系统时所面临的陡峭学习曲线，AI 首要地拉平了谁能启动初创公司或构建产品的竞争场域。

In 2026, a good idea gets founders further than ever. Agentic coding compresses what used to take a team of engineers into work a founder can ship themselves.

2026 年，一个好主意能让创始人走得更远。代理式编码将过去需要工程师团队完成的工作压缩为创始人可自行交付的成果。

The traditional startup growth arc assumes that the path from idea to scale is validate → raise → hire → build → raise again → grow → hire more → repeat.

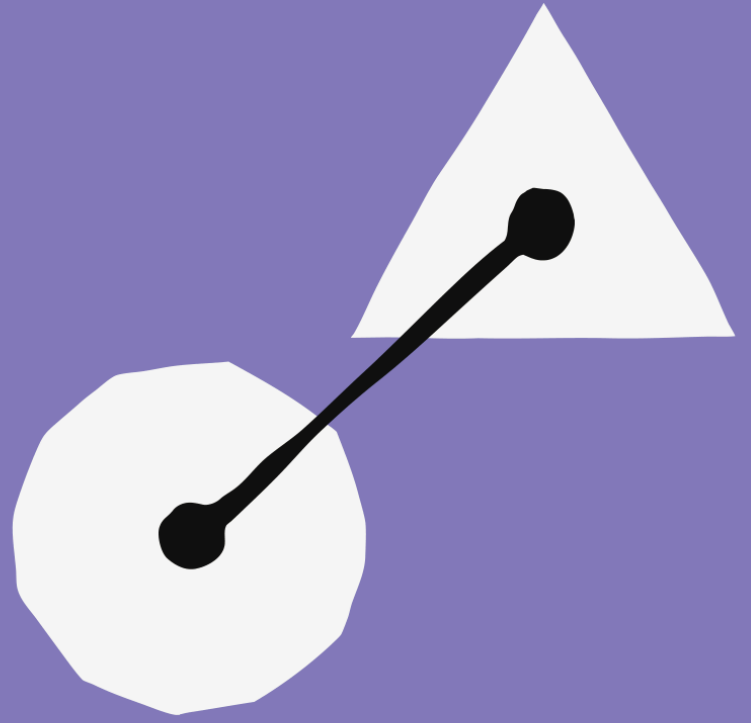
传统初创公司的增长弧线假设从想法到规模化的路径是：验证→融资→招聘→构建→再次融资→增长→更多招聘→重复。

Now, AI has erased the expectation that each new phase in the startup lifecycle requires a bigger team, a different skill set, and a fresh funding round.

如今，AI 消除了初创公司生命周期中每个新阶段都需要更大团队、不同技能组合和新一轮融资的预期。

This playbook remaps the four core stages of the startup journey (Idea, MVP , Launch, and Scale) according to these new realities. W e examine what each stage looks like when AI is core to your technical and organizational development, what the right tools are for each phase, and how founders using these tools are compressing timelines. If you're ready to map the shortest path between idea and exit, read on.

本指南根据这些新现实重新绘制了初创旅程的四个核心阶段（想法、MVP、发布和规模化）。我们探讨当 AI 成为你技术和组织发展的核心时，每个阶段呈现何种面貌，每个阶段所需的合适工具，以及使用这些工具的创始人如何压缩时间线。如果你准备好绘制从想法到退出的最短路径，请继续阅读。



Chapter 2

What it means to be a founder is changing

5

创始人的含义正在改变

What it means to be a founder is changing

Chapter 2 What it means to be a founder is changing Founders used to be defined by what they could do: technical founders wrote code, non-technical founders ran business ops and closed deals. But the models, systems, and AI agents available to founders in 2026 have dissolved the wall between "people who can build" and "people with ideas worth building." AI-native startups are fundamentally transforming what it means to be a founder. Now someone with no engineering background can build production software that brings their idea to life, while a technically adept founder with little business knowledge can easily produce a go-to-market strategy, a financial model, and a highly polished pitch deck.

第二章 创始人含义正在改变 过去，创始人由其能力定义：技术型创始人编写代码，非技术型创始人负责业务运营和促成交易。但 2026 年可供创始人使用的模型、系统和 AI 智能体已消除了“能构建的人”与“值得构建想法的人”之间的壁垒。AI 原生初创公司从根本上改变了成为创始人的含义。如今，没有工程背景的人也能构建生产级软件使其想法落地，而具备技术专长但商业知识有限的创始人也能轻松制定上市策略、财务模型和高度完善的演示文稿。

Historically, founders spent the bulk of their time in execution mode: writing code, managing people, handling day-to-day operational work. In an AI-native startup, the founder role becomes much less individual contributor and much more orchestrator of agents—specialized AI assistants that can read files, run commands, execute code, and even browse the web. The founder's attention shifts up the stack toward the higher-order work: generating ideas and directing the systems (AI agents, tools, and whatever small team exists) that carry those ideas out.

历史上，创始人将大部分时间投入执行模式：编写代码、管理团队、处理日常运营工作。在 AI 原生初创公司中，创始人角色变得更少作为个人贡献者，更多作为智能体的协调者——这些是能够读取文件、运行命令、执行代码甚至浏览网页的专用 AI 助手。创始人的注意力向上转移至更高层级的工作：生成想法并指导承载这些想法的系统（AI 智能体、工具以及任何现有小团队）。

The most revolutionary result of AI as central infrastructure, though, is to unblock non-technical founders with subject matter expertise. When the founding pool expands beyond people with engineering backgrounds, you get startups built by people with radically different lived experiences, solving real problems that the traditional tech-founder pipeline never prioritized (or perhaps even noticed).

然而，AI 作为核心基础设施最革命性的结果是释放了具备专业知识的非技术型创始人。当创始人群体扩展到超越有工程背景的人时，你会看到由拥有截然不同生活经历的人建立的初创公司，解决传统技术创始人管道从未优先考虑（甚至可能未曾注意）的真实问题。

AI tool capabilities for lean startups The traditional startup model assumed you needed to hire engineers to build, salespeople to sell, and ops people to run the business. Headcount was treated as a sign of organizational momentum and product maturity.

AI 工具能力助力精益初创公司：传统初创公司模式假设你需要雇佣工程师来构建、销售人员来销售、运营人员来管理业务。员工人数被视为组织势能和产品成熟度的标志。

Early-stage startups in 2026 are radically different. They're extremely lean by design, often just the founder alone or a team with a few others. By centering both technical and organizational development on AI as infrastructure, they can reach product validation, early revenue, or even profitability before scaling the team. There are three areas in particular where AI helps a startup function like a much larger org: research, agentic coding, and automating workflows for key business operations.

2026 年的早期初创公司截然不同。它们在设计上极其精简，通常只有创始人一人或少数几人组成的团队。通过将技术和组织发展都围绕作为基础设施的 AI 展开，它们可以在扩大团队之前实现产品验证、早期收入甚至盈利。特别是在三个领域，AI 帮助初创公司像规模大得多的组织一样运作：研究、代理式编码以及为关键业务运营自动化工作流。

Conversational intelligence and research Think: on-call expert for every domain Consider everything a founder needs to know in the first year that they almost certainly don't know going in: how do I set up payroll? How do I plan product development sprints? How do I draft a tight investor memo?

对话智能与研究：思考：每个领域的待命专家。思考创始人第一年几乎肯定不知道但需要知道的一切：如何设置薪资？如何规划产品开发冲刺？如何起草一份紧凑的投资者备忘录？

Early-stage startup questions like these all used to have the same answer, which was Find someone who knows. For a bootstrapped or pre-seed founder, this could consume time spent knowledge-gathering instead of building, or possibly requiring burning a chunk of early capital on a consultant. Now, they have AI as an on-call expert across every conceivable domain.

像这样的问题在过去的答案都是“找一位懂行的人”。对于自筹资金或种子轮前的创始人来说，这可能消耗原本用于构建的时间去收集知识，或者可能需要花费早期资本的一小部分聘请顾问。现在，他们在每个 conceivable 领域都有 AI 作为待命专家。

- Deep research: competitive analysis, market sizing, financial modeling
 - 深度研究：竞争分析、市场规模估算、财务建模
-

- Document drafting: pitch decks, case studies, investor memos, PRDs
 - 文档起草：演示文稿、案例研究、投资者备忘录、产品需求文档
-

- Strategic thinking partner: devil's advocate analysis, pre-mortems, scenario planning, roadmap optimization

- 战略思维伙伴：魔鬼代言人分析、事前验尸、情景规划、路线图优化

What it means to be a founder is changing

Agentic coding Think: the engineer who's always available, never blocked Building software used to require a technical co-founder, a contract dev shop, or a long enough runway to hire an engineering team before you'd written a line of production code.

代理式编码：思考：随时待命、从不受阻的工程师。构建软件过去需要一位技术联合创始人、一家合同开发公司，或一段足够长的跑道以便在写出第一行生产代码前雇佣工程团队。

Agentic coding tools now allow every aspiring founder to describe what they want to build in plain language and direct AI to generate, test, debug, and refactor a production-grade codebase at the speed and scale of a full engineering team.

AI 原生编码工具现在允许每一位有抱负的创始人用自然语言描述他们想构建的内容，并直接指令 AI 以完整工程团队的速度和规模生成、测试、调试和重构生产级代码库。

The timeline from “I have an idea” to “I have a product” has compressed. And the founder’s role now centers on what to build and why, while AI handles the actual construction of real infrastructure that’s ready for real users.

从“我有一个想法”到“我有一个产品”的时间线已大幅压缩。创始人的角色现在聚焦于构建什么以及为何构建，而 AI 则负责构建面向真实用户的实际基础设施。

Workflow automation Think: on-demand, automated ops team Even when a founder can research like a consultant and build like an engineering team, there's still a whole category of work beyond strategic planning or product development that still has to get done. Scheduling, updating the CRM, pulling weekly reports, keeping documentation current, publishing content, tracking compliance requirements, managing the connective tissue between the tools and systems the company runs on all have to happen, too. In a lean startup, this load falls mainly on the founder—and it’s a significant tax on the time and attention that should be going toward higher-order decisions.

工作流自动化：思考：按需自动化的运营团队。即使创始人能像顾问一样研究、像工程团队一样构建，仍有一整类工作超出战略规划或产品开发，却必须完成。安排日程、更新 CRM、提取周报、保持文档最新、发布内容、跟踪合规要求、管理公司运行所依赖的工具与系统之间的连接纽带，都必须发生。在精益初创公司中，这一负担主要落在创始人身上——这是对本应用于更高层级决策的时间和注意力的重大消耗。

Workflow automation with AI tools offloads that tax. Recurring operational tasks can be configured to happen automatically so that the CRM updates when a deal moves, a weekly report compiles itself, and product documentation gets updated in sync with product changes. And, crucially, Claude Cowork integrates with the interconnected systems a startup runs on—your project management tool, your communication stack, your data sources—without needing someone to build and maintain those integrations. In Day Zero startups, that someone is almost always the founder.

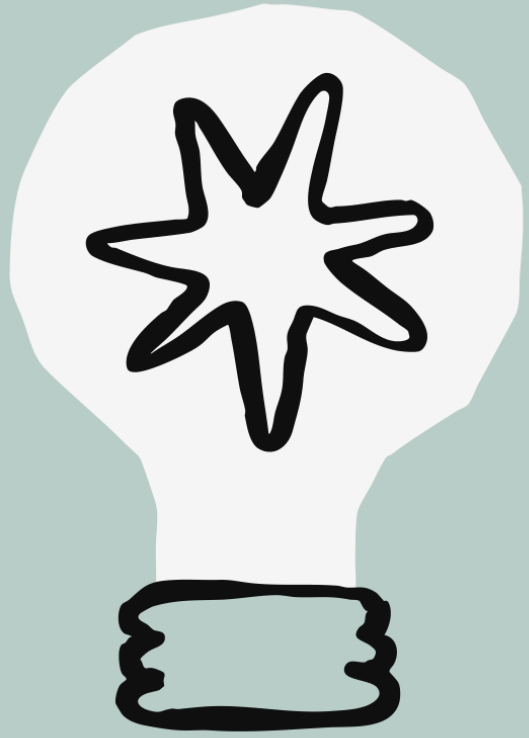
借助 AI 工具的工作流自动化可转移这一负担。重复性运营任务可配置为自动执行，例如交易推进时自动更新 CRM、周报自动汇总、产品文档随产品变更同步更新。关键的是，Claude Cowork 与初创公司运行的互联系统（你的项目管理工具、沟通栈、数据源）集成，无需有人专门构建和维护这些集成。在 Day Zero 初创公司中，这个人几乎总是创始人。

Timing and orchestration are everything Founders that effectively harnesses AI's research, automation, and agentic coding capabilities can build a startup that operates with far more leverage than its headcount suggests. They also get to dedicate the majority of their time and bandwidth to the work that actually matters.

时机与协调至关重要：有效利用 AI 的研究、自动化和代理式编码能力的创始人，能够构建一家运营杠杆远超其员工人数所显示的初创公司。他们也能将大部分时间和带宽投入到真正重要的工作中。

This work doesn't happen on autopilot; the founder orchestrating these AI tools needs to know how (and when) to apply them. The rest of this playbook is dedicated to exploring the goals and challenges founders will encounter as they follow the AI-native startup path, and how to effectively apply AI tools at each stage of the journey.

这项工作并非自动完成；编排这些 AI 工具的创始人需要知道如何（以及何时）应用它们。本手册的其余部分致力于探讨创始人在遵循 AI 原生初创公司路径时将遇到的目标与挑战，以及如何在旅程的每个阶段有效应用 AI 工具。



Chapter 3

Idea Stage

Idea Stage

Chapter 3 Idea Stage Every startup founder starts from the same place: a problem they can't stop thinking about. This is the startup phase where idea meets reality: startup success in 2026 requires the discipline of not building until the evidence justifies it.

[第3章 构思阶段] 每位初创公司创始人都是从同一个起点开始：一个让他们无法停止思考的问题。这是构思与现实的交汇阶段：2026 年的初创公司成功要求具备一种纪律，即在证据足以证明其合理性之前不急于构建。

The work in this stage is research, customer discovery, competitive analysis, and honest evaluation of disconfirming evidence, all before asking Claude Code to generate your first line of production code.

"The work in this stage is research, customer discovery, competitive analysis, and honest evaluation of disconfirming evidence, all before asking Claude Code to generate your first line of production code." -> "本阶段的工作包括研究、客户发现、竞争分析以及对反驳证据的诚实评估，所有这些都在要求 Claude Code 生成你的第一行生产代码之前完成。"

Idea stage goal While in the Idea stage, the founder's main goal is research-oriented validation:

构思阶段目标 在构思阶段，创始人的主要目标是研究导向的验证：

assembling solid evidence that a real problem exists (and that your proposed solution effectively addresses it) before committing resources to building.

"assembling solid evidence that a real problem exists (and that your proposed solution effectively addresses it) before committing resources to building." -> "在投入资源进行构建之前，收集确凿证据证明真实问题存在（且你提出的解决方案能有效解决它）。"

Practically speaking, the Idea stage is a series of questions a founder has to answer in roughly this order:

实际上，构思阶段是一系列创始人必须按大致此顺序回答的问题：

-
- Is this problem real, specific, and frequent enough to build around?
 - 这个问题是否真实、具体且足够频繁，值得围绕它构建？
-

- Who exactly has it, and is that a market?
 - 究竟是谁拥有它，这是否构成一个市场？
-

- Is anyone else solving it, and if so, how and how well?
 - 是否有人正在解决它，如果是，他们如何解决以及解决得有多好？
-

- What would a solution actually need to do in order to solve this problem, and does my idea do that?
 - 解决方案实际上需要做什么才能解决这个问题，我的想法能做到吗？
-

The results of these inquiries add up to answer a single, ultimate question: Is this worth building?

这些调查的结果汇总起来，旨在回答一个单一的终极问题：这值得构建吗？

That means getting specific before you get moving. "People struggle with expense reporting" is an observation. "Finance managers at mid-market companies spend four-plus hours a week reconciling submissions because their current tools don't integrate with their accounting software" is a testable hypothesis.

这意味着在行动之前要具体化。“人们难以报销费用”是一个观察结果。“中型公司的财务经理每周花费四小时以上对账，因为他们的现有工具与会计软件不集成”是一个可验证的假设。

Idea stage exit criteria The Idea stage exit condition is finding problem-solution fit. You've established qualitative evidence, primarily from real human conversations, that you're solving a real problem for real people before you start building the thing that solves it.

构思阶段退出标准 构思阶段的退出条件是找到问题 - 解决方案匹配。在开始构建解决该问题的产品之前，你已通过主要来自真实人类对话的定性证据，确立了你在为真实人群解决真实问题。

You're ready to leave the Idea stage when you can answer yes to all three of the following:

当你能够对以下三个问题都回答“是”时，就准备好离开创意阶段了：

1. Is the problem real and specific? Answering in the affirmative here requires that you can name exactly who experiences this problem, how often they encounter it, how severely it affects them, and what they currently do about it.

1. 问题是真实且具体的吗？在此肯定回答要求你能明确指出谁经历这个问题、他们多久遇到一次、问题对他们的影响程度，以及他们目前如何应对。

2. Does your solution address the actual problem? Not the problem you originally assumed, but the one the validation process revealed. Sometimes these are the same thing, but not always.

2. 你的解决方案是否解决了实际问题？不是最初假设的问题，而是验证过程揭示的问题。有时两者相同，但并非总是如此。

3. Do you have enough signal to justify building? You will never have certainty at this stage, and waiting for it is its own failure mode, but you need enough qualitative evidence that committing to an MVP is a reasoned decision over an act of faith.

3. 你是否有足够的信号来证明构建的合理性？在此阶段你永远不会拥有确定性，而等待确定性本身就是一种失败模式，但你仍需要足够的定性证据，以证明承诺构建最小可行产品（MVP）是一个经过深思熟虑的决定，而非一种信仰之举。

Idea stage challenges The Idea stage is where the most important work of your startup journey happens, because it's where the most consequential mistakes are made: getting something wrong now can quickly run your budding venture right off the rails.

构思阶段挑战 构思阶段是你初创公司旅程中最重要的工作发生之处，因为这里最容易犯下后果最严重的错误：现在犯下的错误会迅速让你的初创企业脱轨。

The majority of ideation phase challenges involve moving faster than your understanding justifies, though, so founders who proceed with thoughtfulness and deliberation will experience steady progress.

然而，构思阶段的大多数挑战都涉及行动速度快于理解所支持的程度，因此那些以审慎和深思熟虑推进的创始人将体验到稳步进展。

Idea Stage

Mistaking building for validating The challenge: When technical blockers are lifted, an impassioned founder risks skipping the most important work in the startup journey: validating that their idea is genuinely a solution that people need and will use.

误将构建当作验证 挑战：当技术障碍被清除后，充满激情的创始人可能会跳过初创公司旅程中最重要的一步：验证他们的想法是否确实是人们需要并会使用的有效解决方案。

Even before the current era of agentic coding, 42% of startups failed because they built something nobody wanted. Now, though, agentic coding solutions like Claude Code have drastically collapsed the distance between "I have an idea" and "I have a product" and that failure rate is only going to climb.

即使是在当前代理式编码时代之前，42% 的初创公司就因构建了无人想要的产品而失败。如今，像 Claude Code 这样的代理式编码解决方案已大幅缩短了从“我有一个想法”到“我有一个产品”的距离，而这一失败率只会进一步攀升。

While there's never been a better time to be a founder with a synapse-shakingly good idea, the rapidity and ease of spinning up a prototype that looks something like a product also, counterintuitively, presents a genuinely dangerous existential risk for the AI-native startup.

虽然对于拥有令人震撼的好想法的创始人来说，这或许是最好的时代，但快速且轻松地构建出类似产品的原型，也反直觉地为 AI 原生初创公司带来了真正危险的生存风险。

Until very recently, building required real dev time and budget, and getting even a basic prototype together typically took months. Now that the hurdle of technical development is largely gone, though, AI makes it all too easy for a founder to jump straight into building without validating its utility in the real world.

直到最近，构建仍需真实的开发时间和预算，通常搭建一个基本原型需要数月时间。如今，随着技术开发的门槛基本消失，AI 让创始人过于容易地直接跳入构建阶段，而无需验证其在现实世界中的效用。

Reaching problem-solution fit requires first validating your hypothesis then building, but many first-time (and even experienced) founders mistakenly believe that AI short-circuits that requirement, turning the flow into have an idea -> immediately build a prototype -> treat the existence of the prototype as validation. The prototype becomes a reason to believe the hypothesis was right all along, without ever testing whether it's actually true.

达到问题 - 解决方案匹配度需要先验证假设再构建，但许多首次（甚至是有经验的）创始人错误地认为 AI 短路了这一要求，将流程转变为：有一个想法 -> 立即构建原型 -> 将原型的存在视为验证。原型成了相信假设一直正确的理由，却从未测试其是否真正成立。

A working prototype is easy to mistake as concrete evidence that you're solving a real problem, but it's not. Your prototype instead serves as a useful pressure- testing prop for conversations with potential users. These conversations themselves are the real evidence.

一个可工作的原型容易被误认为是解决真实问题的具体证据，但事实并非如此。你的原型反而可作为与潜在用户对话时有用的压力测试道具。这些对话本身才是真正的证据。

Premature scaling The challenge: When building is effortless and instant, you can scale execution far ahead of what business demands.

过早扩张 挑战：当构建变得 effortless 且即时时，你的执行扩张速度可能会远远超过业务需求。

Scaling prematurely means committing to a product path before you've genuinely validated that the path is worth committing to.

过早扩张意味着在你尚未真正验证该路径值得投入之前，就承诺了一条产品路线。

This has always been a startup killer, but AI has made it dramatically easier for founders to fall into the premature scaling trap without noticing. Agentic coding assistants are so powerful that it's easy to scale execution far ahead of validating problem-solution fit without ever consciously deciding to stray off course.

这向来是初创公司的杀手，但 AI 让创始人更容易在不察觉的情况下陷入过早扩张的陷阱。代理式编码助手如此强大，以至于很容易在验证问题 - 解决方案匹配度之前就大幅扩张执行规模，而无需有意识地偏离正轨。

It will generate, test, debug, and refactor a codebase around a fundamentally flawed premise with exactly the same enthusiasm it brings to a great idea. The intelligence in the system is yours. The prime directive at this stage is keeping your sense-making ahead of your building, especially when building is so quick and feels so effortless.

它将以对待伟大想法同样的热情，围绕一个根本有缺陷的前提生成、测试、调试和重构代码库。系统的智能归你所有。此阶段的首要指令是让你的意义构建能力领先于你的构建能力，尤其是在构建如此迅速且感觉毫不费力时。

Loss of objectivity The challenge: Ask an AI tool for evidence supporting what you already believe, and it will find it. Confirmation bias now comes with a research engine.

1. 失去客观性 挑战：向 AI 工具询问支持你已有信念的证据，它会找到它。确认偏误现在附带了一个研究引擎。

Confirmation bias has always been an occupational hazard in startups:

确认偏误一直是初创公司的职业风险：

founders are, by nature, passionate about their ideas. Now, AI tools have given confirmation bias a significant powerup. Ask AI to validate your startup idea and it will find supporting evidence; ask it to size your potential market and it will find the number that makes your TAM look fundable.

创始人天生对自己的想法充满热情。现在，AI 工具让确认偏误获得了显著增强。让 AI 验证你的初创公司想法，它会找到支持性证据；让 AI 估算你的潜在市场规模，它会找到让你的 TAM 看起来可融资的数字。

AI follows your direction, which means a founder who isn't asking hard questions can now construct an elaborate, well-researched-looking case for a bad idea faster than ever before, while feeling fully confident that they are, in fact, performing due diligence. The antidote is the same tool, only pointed in the opposite direction: AI will pressure-test an idea just as thoroughly as it validates one.

AI 跟随你的方向，这意味着一个不提出棘手问题的创始人，现在比以往任何时候都能更快地构建一个精心、看似经过充分研究的坏想法案例，同时完全自信地认为自己实际上正在履行尽职调查。解药是同一个工具，只是指向相反的方向：AI 验证一个想法的彻底程度，同样也能对想法进行压力测试。

When research and structured adversarial thinking surface evidence that your idea needs revision, this the signal to pivot.

当研究和结构化的对抗性思维浮现出你的想法需要修订的证据时，这就是转向的信号。

Idea Stage

How Claude can help Idea stage founders Progressing your AI-native startup concept through the Idea stage can feel like it takes forever . You are a founder and you just want to build. But this all-important kickoff phase is fundamentally a research and validation exercise, which means reaching for the tools that help you think more rigorously before going all in on writing code. Here are ways to use Claude across its product surfaces (Chat, Claude Cowork, and Claude Code) for moving through the Idea stage as quickly as possible while doing proper due diligence.

2. Claude 如何帮助 创意阶段创始人 将你的 AI 原生初创公司概念推进到创意阶段，可能会感觉耗时漫长。你是一名创始人，只想开始构建。但这个至关重要的启动阶段本质上是一项研究和验证练习，这意味着在全力投入编写代码之前，先使用那些帮助你更严谨思考的工具。以下是利用 Claude 在其产品界面（Chat、Claude Cowork 和 Claude Code）上的方式，以便在尽到适当尽职调查的同时，尽可能快地通过创意阶段。

Chat, Claude Cowork, or Claude Code:

Chat、Claude Cowork 或 Claude Code：

choosing the right Claude surface

选择正确的 Claude 界面

AI makes it easier for startup founders to ship faster, automate tedious workflows, and operate at scale, but the surface you use matters. Here's when to use Chat, Claude Cowork, or Claude Code depending on the task at hand.

AI 使初创公司创始人能更快发布产品、自动化繁琐 workflows 并实现规模化运营，但所使用的界面至关重要。此处根据手头任务说明何时使用 Chat、Claude Cowork 或 Claude Code。

Chat is for quick exchanges without leaving the app you're already in. Use it for the constant small tasks of running a company: pulling the one-sentence takeaway from a dense investor memo, sanity-checking a claim before a board meeting, or making sense of a long Slack thread with your team.

Chat 适用于在不离开当前应用的情况下进行快速交流。用于公司运营中的持续小任务：从密集的投资备忘录中提取一句话要点、在董事会会议前核实主张、或与团队梳理冗长的 Slack 线程。

Claude Cowork is for the knowledge work that actually takes time: pulling from many sources, making sense of it, and producing something finished, like a doc, deck, or spreadsheet. Think turning a folder of customer call transcripts into a themed findings doc for your next product review, building a competitive landscape from a dozen vendor sites before a fundraise, or a standing Monday-morning task that pulls metrics from your connected tools and drops a weekly KPI brief into a shared folder.

Claude Cowork 适用于真正需要时间的知识工作：从多个来源提取信息、整合理解并产出成品，如文档、演示文稿或电子表格。例如将客户通话录音文件夹转化为下次产品评审的主题发现文档、在融资前从 dozen 个供应商网站构建竞争格局，或执行每周一早晨的固定任务，从连接工具中提取指标并将每周 KPI 简报放入共享文件夹。

Claude Code is the agentic coding environment for the engineers on your team: direct codebase access, Plan Mode, git integration, and local, IDE, or sandboxed cloud environments. It's where a lean team ships features across a growing codebase, migrates legacy code from the MVP days, and moves from prototype to production without waiting on more headcount.

Claude Code 是团队工程师的代理编码环境：直接访问代码库、Plan 模式、git 集成，以及本地、IDE 或沙盒云环境。它是精简团队在增长中的代码库上发布功能、从 MVP 时代迁移遗留代码、并在不等待更多人手的情况下从原型过渡到生产的地方。

If the task is... Reach for Why A question, a rewrite, a quick brainstorm Chat Fast, conversational, no setup Research, analysis, or a finished document built from your files and systems Claude Cowork Folder access, connectors, skills, scheduled runs Writing, testing, or shipping software Claude Code Codebase access, diffs, git, dev environments

3. 如果任务是... 为何要使用 问题、改写、快速头脑风暴 Chat 快速、对话式、无需设置 研究、分析，或基于你的文件和系统构建的完整文档 Claude Cowork 文件夹访问、连接器、技能、计划运行 编写、测试或发布软件 Claude Code 代码库访问、差异、git、开发环境

The three share the same Claude underneath; what changes is the workspace around it.

这三个共享底层的 Claude；变化的是围绕它的workspace。

Defining and pressure-testing the problem hypothesis Your own domain expertise and up-front research have already generated a hypothesis. The first job is to sharpen it until it's actually testable. Claude is particularly useful here for forcing specificity: who exactly has this problem, how often, how severely, and what do they currently do about it? A problem statement that can't answer those questions precisely isn't ready to validate.

4. 定义和压力测试问题假设 你自身的领域专业知识和前期研究已经生成了一个假设。首要任务是将其打磨至实际上可测试的程度。Claude 在此特别有用，因为它能迫使具体化：究竟是谁有这个问题，频率如何，严重程度如何，以及他们目前对此做了什么？一个无法精确回答这些问题的陈述尚未准备好进行验证。

- Exercise: Work with Claude to sharpen your problem statement until it's a testable hypothesis. For example, "Contract review takes too long" is not meaningfully testable. But "In-house legal teams at mid-market companies spend 3+ days per contract review cycle because redlines are managed across email threads rather than a single version-controlled document" is very testable.

- 练习：与Claude协作打磨您的问题陈述，直至成为可测试的假设。例如，“合同审查耗时过长”并非有意义可测试。但“中型公司的内部法务团队因红线在邮件线程而非单一版本控制文档中管理，导致每个合同审查周期花费3天以上”则非常可测试。

Your next move is to ask Claude to argue against your idea, and to find disconfirming evidence that refutes your hypothesis. This can surface negative market signals, failed competitors, customer behavior patterns, and structural obstacles that a supportive synthesis would have quietly deprioritized.

您的下一步是请Claude反驳您的想法，并寻找否定您假设的反证。这能揭示负面市场信号、失败的竞争对手、客户行为模式以及支持性综合可能悄悄降低优先级的结构性障碍。

Idea Stage

The goal is to arrive at customer discovery having already stress-tested your assumptions against the strongest available counterarguments so that informational user interviews are genuinely open-ended rather than a search for confirmation.

目标是带着客户发现，已用最强有力的可用反论对您的假设进行了压力测试，从而使信息性用户访谈真正开放，而非寻找确认。

Note: Using Claude as structured devil's advocate is a core use case at every stage of the AI startup life cycle.

注意：将 Claude 作为结构化的魔鬼代言人，是 AI 初创公司生命周期每个阶段的核心用例。

Market research and mapping the competitive landscape Sizing up your competitors There's a startup-specific phenomenon called competitor neglect: the tendency to focus so intensely on your own vision and execution that you systematically underweight what others are doing in the same space. Fortunately, AI offers the antidote: ask Claude to make the most compelling argument for why a competitor in this solution space would succeed while you do not.

5. 市场研究和绘制竞争格局 评估你的竞争对手 有一个特定的初创公司现象称为竞争忽视：倾向于如此强烈地关注你自己的愿景和执行，以至于系统性地低估同一领域其他人的所作所为。幸运的是，AI 提供了对策：让 Claude 提出一个最有力的论点，解释为什么在这个解决方案领域的竞争对手会成功，而你会失败。

Claude can analyze why their approach is actually better, why customers would choose them, why your potential differentiators may not be as defensible as you think.

Claude 可以分析为何他们的方案实际上更好、为何客户会选择他们、为何你们潜在的差异化优势可能不如你们认为的那样具有防御性。

- **Exercise:** Ask Claude to map your competitive landscape by tier: direct competitors, indirect competitors, potential acquirers, and adjacent players who could move into your space. Then ask it to argue for why each tier poses a genuine threat to your success, not just the version of the threat that's easiest to dismiss.

- **练习：**让 Claude 按层级映射你的竞争格局：直接竞争对手、间接竞争对手、潜在收购方，以及可能进入你领域的相邻参与者。然后让它论证为何每个层级都对你们的成功构成真正威胁，而不仅仅是那些最容易驳回的威胁版本。

Market research Claude Code can synthesize publicly available customer feedback to surface recurring complaints and unmet needs. Bonus: doing this is essentially free qualitative research on your competitors' customers.

6. 市场研究 Claude Code 可以综合公开可用的客户反馈，以浮现 recurring complaints 和未满足的需求。额外好处：这样做本质上是对竞争对手客户进行的免费定性研究。

- Exercise: Direct Claude Cowork to synthesize competitor reviews across your key sources and identify the top complaints that existing solutions haven't resolved. If your hypothesis addresses one or more of them, that's strong evidence of problem-solution fit. If it doesn't, that's worth knowing too.

- 练习：直接让 Claude Cowork 综合关键来源的竞争对手评论，并识别现有解决方案尚未解决的主要 complaints。如果你的假设解决了一个或多个问题，那就是问题 - 解决方案匹配的有力证据。如果没解决，这也值得知晓。

Claude Cowork can also extract relevant information and figures from dense industry reports, analyst filings, and market research documents; next, these clean, synthesized inputs become ideal context for Claude's analysis work.

Claude Cowork 还可以从密集的行业报告、分析师申报和市场研究文件中提取相关信息和数字；接下来，这些干净、综合的输入成为 Claude 分析工作的理想上下文。

- Exercise: Build TAM/SAM/SOM models from publicly available data and pressure-test the assumptions behind them. Identify whether the market is expanding, consolidating, or mature; this context influences how you think about timing and differentiation. Map the buyer landscape: who holds budget, who influences decisions, and whether those are the same person.

- 练习：基于公开数据构建 TAM/SAM/SOM 模型并压力测试其背后的假设。识别市场是扩张、整合还是成熟；此上下文影响你如何思考时机和差异化。描绘买家格局：谁持有预算，谁影响决策，以及这些人是否为同一人。

Trend analysis Finally , use Claude to listen for early indicators that tell you whether you're entering at the right moment. Track subreddits and LinkedIn groups where conversations about your problem are already happening and the exact language users reach for when describing their issues. Ask Claude to identify analogous markets where a similar problem was solved, and extract what worked and what didn't. Surface regulatory , technological, or demographic trends that could accelerate or threaten the opportunity.

趋势分析：最后，利用 Claude 倾听早期指标，以判断你进入的时机是否正确。追踪那些关于你所在问题的对话已经发生的子版块和 LinkedIn 群组，以及用户在描述问题时使用的确切语言。询问 Claude 识别类似市场，其中类似问题已被解决，并提取哪些有效、哪些无效。揭示可能加速或威胁该机会的监管、技术或人口趋势。

- Exercise: Ask Claude to identify three external trends—regulatory , technological, or demographic—that could significantly affect your market in the next two years, and to assess whether each one is a tailwind or a headwind for your specific hypothesis.

- 练习：请 Claude 识别三个外部趋势——监管、技术或人口——这些趋势可能在未来两年内显著影响您的市场，并评估每个趋势是顺风还是逆风，针对您的具体假设。

Note: The market research and competitive mapping work in this section isn't a one-time exercise. You are going to continue making discoveries and evolving your thinking through the MVP and Launch stages, so it's important to repeat these exercises whenever your hypothesis evolves.

注意：本部分的市场研究和竞争映射工作不是一次性的练习。您将在 MVP 和发布阶段继续做出发现并演进您的思维，因此每当您的假设演进时，重复这些练习非常重要。

Plan and design customer discovery The quality of what you learn by talking to potential users for your product is determined by (1) the quality of the questions you ask and (2) whether you are posing these to the right people. Claude is particularly helpful for conducting customer discovery, including who to talk to, what to ask, and how to make sense of what you hear.

计划和设计客户发现：通过与潜在用户交谈所获学习的质量，取决于 (1) 你提问的质量，以及 (2) 你是否向正确的人提问。Claude 在客户发现方面特别有帮助，包括与谁交谈、问什么，以及如何理解你所听到的内容。

Idea Stage

Who to talk to A precise target profile is infinitely more valuable than a long contact list, including specific job titles, company types, team structures, and seniority levels most likely to experience the problem acutely. From there, identify where those people are actually reachable—the communities, events, LinkedIn groups, and Slack workspaces where they congregate—and build a prioritization framework for who to reach out to first based on how close they are to the problem.

与谁交谈：精确的目标画像比长长的联系人名单更有价值，包括最有可能深切体验该问题的具体职位、公司类型、团队结构和资历水平。在此基础上，确定这些人实际可触及的地方——他们聚集的社区、活动、LinkedIn 群组 and Slack 工作区——并建立一个优先排序框架，根据他们与问题的接近程度决定首先联系谁。

What to ask With your targets defined, use Claude to build the interview framework itself:

问什么：在目标已定义后，利用 Claude 构建访谈框架本身：

the right questions, in the right order, structured to surface what people actually do rather than what they think they would do. A rookie founder mistake is asking a generic, open-ended question about the future ("would you use something like this?") instead of specifically querying the relevant past ("tell me about the last time you dealt with this problem.")

提出正确的问题，按正确的顺序，结构化地揭示人们实际在做什么，而非他们认为会做什么。新手创始人常犯的错误是询问关于未来的通用开放式问题（“你会使用类似的东西吗？”），而不是具体询问相关的过去（“告诉我上次你处理这个问题的经历。”）

Claude can flag where your draft questions are leading the respondent, too broad, or otherwise likely to generate noise instead of signal. Claude can also help you in designing follow-up questions to probe deflections or drill down on vague answers to important questions.

Claude 还能标记你的草稿问题是否引导受访者、过于宽泛，或更可能产生噪音而非信号。Claude 还能帮你设计后续问题，以探究回避或深入挖掘重要问题上的模糊回答。

If your hypothesis involves more than one persona, Claude can also design different question sets for each. A finance manager and a CFO have different relationships to the same problem, and a single interview framework will flatten that distinction.

如果你的假设涉及多个用户画像，Claude 还能每个画像设计不同的问题组。财务经理和 CFO 对同一问题的关系不同，单一的访谈框架会抹平这种区别。

- Exercise: Draft your interview questions by hand first, ask Claude to audit them. Ask it specifically to flag any question that is leading, future-facing, too broad, or likely to produce a socially desirable answer rather than an honest one. Then ask it to suggest a follow-up probe for the two or three moments in the interview most likely to generate deflection.

- 练习：先用笔起草你的面试问题，然后让 Claude 审核它们。具体请它标记任何引导性强、面向未来、过于宽泛，或更可能产生社会期望答案而非诚实答案的问题。接着，请它为面试中最可能产生回避反应的两三个时刻建议后续追问。

Post-interview analysis After each conversation, use Claude to debrief: feed it your notes and ask it to identify what confirmed your hypothesis, what challenged it, and what was genuinely surprising. Once you've gathered a batch of interviews, run your full set of interview notes through Claude Cowork to surface recurring themes, contradictions, and the strongest signals in both directions. Then take that synthesized output back to Claude and ask it to flag where your own read of the data might be pattern-matching to what you want to hear rather than what's actually there.

访谈后分析：每次对话后，利用 Claude 进行复盘：将你的笔记输入其中，并询问它哪些内容证实了你的假设、哪些挑战了它、以及哪些真正令人惊讶。一旦收集到一批访谈记录，将整套访谈笔记通过 Claude Cowork 处理，以揭示 recurring themes（recurring themes）、矛盾以及正反两方面的最强信号。然后将该综合输出带回 Claude，并询问它标记出你自己的数据解读可能在何处是模式匹配到了你想听到的内容，而非实际存在的内容。

- Exercise: After every five interviews, direct Claude Cowork to synthesize your notes and produce two lists: the evidence that supports your hypothesis, and the evidence that challenges it. If the first list is significantly longer than the second, ask Claude whether that asymmetry reflects what's actually in the data—or what you were hoping to find.

- 练习：每完成五次访谈，指示 Claude Cowork 综合你的笔记并生成两份清单：支持你假设的证据，以及挑战你假设的证据。如果第一份清单显著长于第二份，请 Claude 判断这种不对称性反映的是数据中的实际情况，还是你希望找到的情况。

Customer outreach and scheduling Use Claude Cowork to automate the operational lift around building a contact list, running outreach, and scheduling user interviews.

客户触达与排期：利用 Claude Cowork 自动化围绕建立联系人名单、进行触达和安排用户访谈的运营工作。

Claude Cowork can use the target profile you defined with Claude (including job titles, company types, and seniority levels) to research and compile a structured list of prospects and verified contact information. It then drafts personalized outreach emails at scale, tailoring each one to the individual's role and context.

Claude Cowork 可以利用您用 Claude 定义的目标档案（包括职位头衔、公司类型和职级水平）来研究和编译一份结构化的潜在客户列表及已验证的联系信息。随后，它大规模起草个性化的外联邮件，并根据每个个体的角色和背景进行定制。

As responses come in, it connects to Gmail and Google Calendar via MCP to manage the thread, handle scheduling requests, and get interviews on the calendar . The workflow continues as Claude Cowork generates follow-up drafts on a defined cadence (a day-seven follow-up for contacts who haven't responded, for instance) and updates your tracking sheet as each step completes so you always know where every prospect stands in the pipeline.

随着回复的到达，它通过 MCP 连接 Gmail 和 Google Calendar 来管理对话线程、处理日程安排请求，并将面试安排到日历中。工作流程持续进行，Claude Cowork 按既定节奏生成跟进草稿（例如，对未回复的联系人进行第 7 天跟进），并在每一步完成后更新您的跟踪表，以便您始终了解每位潜在客户在管道中的状态。

- **Exercise:** Give Claude Cowork your validated interview target profile and ask it to build a prospect list, draft a personalized outreach sequence, and set up a tracking sheet with columns for outreach status, follow-up cadence, and interview completion. Then let it run the coordination while you focus on preparing for the conversations themselves.

- **练习：**向 Claude Cowork 提供您已验证的面试目标档案，要求它构建潜在客户列表、起草个性化外联序列，并设置包含外联状态、跟进节奏和面试完成情况的列的跟踪表。然后让它运行协调工作，而您专注于准备对话本身。

Idea Stage

Design your final solution concept You've done the validation work: the problem is real, you know who has it, and you have a solution concept that the evidence supports. Use Claude to develop and challenge your solution concept from every angle: What are the gaps? What alternatives exist? What would have to be true for this solution to work at scale ?

设计你的最终解决方案概念：你已经完成了验证工作：问题真实存在，你知道谁有问题，并且有一个有证据支持的解决方案概念。使用 Claude 从各个角度开发和挑战你的解决方案概念：存在哪些差距？有哪些替代方案？要使该解决方案能够规模化，必须满足什么条件？

This is an important reality checkpoint: does this design actually address the problem the validation process revealed, and not the problem you originally assumed going in?

这是一个重要的现实检查点：该设计是否真正解决了验证过程揭示的问题，而不是您最初假设的问题？

-
- Exercise: Present your solution concept to Claude and ask it to identify the three assumptions your design depends on most heavily. Then ask what would have to be true for each assumption to hold, and what the consequences are if any one of them doesn't.
 - 练习：向 Claude 展示你的解决方案概念，并请它识别你的设计最依赖的三个假设。接着询问每个假设成立需要满足什么条件，以及如果其中任何一个不成立会有什么后果。

Build a lightweight prototype with Claude Code Now for the fun part: with a validated hypothesis and a stress-tested solution concept, you're finally ready to build something.

使用 Claude Code 构建轻量级原型：现在到了有趣的部分：有了经过验证的假设和经过压力测试的解决方案概念，你终于准备好构建某样东西了。

This is the moment in the Idea stage where Claude Code enters the picture. Even if you've been tinkering all along, now is the point where you generate your official lightweight prototype: the minimum surface area needed to put your idea in front of a real human and get a genuine reaction.

这是“想法”阶段中 Claude Code 介入的时刻。即使你一直在 tinkering，现在也是生成你的官方轻量级原型的节点：这是将你的想法呈现给真实人类并获得真实反应所需的最小表面积。

You're not building a real-world product (yet); you're constructing a functional sample of your idea to use in customer and investor conversations. Real users reacting to something they can actually touch will tell you things that a dozen problem-solution discovery interviews couldn't. Before, you were establishing that the problem you're solving is real; now, you are asking potential users to engage with the proposed solution.

你（尚未）在构建现实世界中的产品；你是在构建你想法的功能性样本，用于客户和投资者对话。真实用户对可触摸事物的反应，会告诉你 dozen 次问题 - 解决方案发现访谈无法告诉你的事情。此前，你是在确立你正在解决的问题是真实的；现在，你是在让潜在用户与提出的解决方案互动。

- Exercise: Define the single core interaction your solution depends on. Direct Claude Code to build only that. When you have it, put it in front of five people from your validated target profile and ask them to try it out. What you learn in those five conversations determines whether you keep building, or go back to the drawing board.

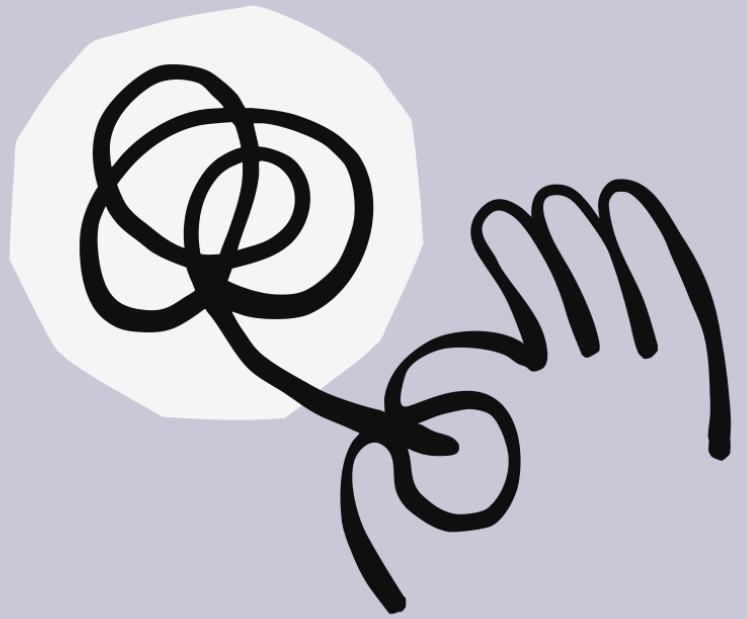
- 练习：定义你的解决方案所依赖的单一核心交互。指示 Claude Code 仅构建该部分。当你拥有它时，将其展示给你验证过的目标画像中的五个人，并请他们试用。那五次对话中学到的内容决定了你是继续构建，还是回到起点重做。

Reaching the end of the Idea stage is a giant leap ahead in the AI startup race because now you're not betting on a hunch; you're executing against evidence.

到达创意阶段的终点是 AI 初创公司竞赛中的巨大飞跃，因为现在你不再押注于直觉，而是依据证据进行执行。

Now comes the MVP stage, where the founder's guiding question goes from "Is this worth building?" to "What exactly should we build first?" and AI's primary role shifts from research partner to construction crew.

接下来是 MVP 阶段，创始人的核心问题从“这值得构建吗？”转变为“我们究竟应该先构建什么？”，而 AI 的主要角色从研究伙伴转变为施工团队。



Chapter 4

MVP Stage

15

MVP 阶段

MVP stage

Chapter 4 MVP Stage Plenty of founders treat the MVP stage as a construction phase, but the MVP stage is still fundamentally an evidence-gathering exercise. The difference is that you are now gathering evidence about the solution instead of the problem space;

第 4 章 MVP 阶段：许多创始人将 MVP 阶段视为建设阶段，但 MVP 阶段本质上仍是一个收集证据的练习。区别在于，你现在收集的是关于解决方案的证据，而不是关于问题空间的证据；

specifically , whether a real, identifiable group of people finds it valuable enough to use it, return to it, pay for it, and/or tell others about it.

具体来说，就是看一个真实且可识别的人群是否认为它足够有价值，从而使用它、回访它、付费购买它和/或向他人推荐它。

MVP stage goals As the founder of an AI-native startup, your goal is to translate a validated problem into a working product that real users will actually use. This is not the full version with every roadmap feature but the smallest, most focused iteration of your idea that puts a real solution in front of real users and generates real evidence of product-market fit.

MVP 阶段目标：作为 AI 原生初创公司的创始人，你的目标是将经过验证的问题转化为一个实际用户真正会使用的工作产品。这不是包含所有路线图功能的完整版本，而是你想法中最小、最专注的迭代版本，它将真正的解决方案呈现给真正的用户，并生成关于产品与市场契合度的真实证据。

At the same time, how you build now determines what's possible later . This means that the MVP stage has a second, equally important goal of moving fast without accruing the type of technical debt that compounds—and will haunt you the moment real users arrive in meaningful numbers.

同时，你现在的构建方式决定了未来的可能性。这意味着 MVP 阶段还有第二个同样重要的目标：快速推进，同时不积累那种会复利增长——并在真实用户以有意义数量到来时立刻困扰你的技术债务。

And finally , investing in persistent context from day one is what keeps AI a force multiplier instead of a source of entropy. In an AI-native startup, your codebase is something you collaborate with AI on session after session, which makes legibility foundational. Founders who skip specs, architectural decisions, and context files (like CLA UDE.md) hit a predictable wall where every new session requires re-explaining the codebase and AI-generated changes drift from the original vision.

最后，从第一天起就投资持久上下文，是让 AI 成为倍增器而非熵源的途径。在 AI 原生初创公司中，你的代码库是你与 AI 逐次协作的对象，这使得可读性成为基础。跳过规范、架构决策和上下文文件（如 CLAUDE.md）的创始人会撞上一堵可预见的墙，即每次新会话都需要重新解释代码库，而 AI 生成的更改会偏离原始愿景。

MVP stage exit criteria The MVP stage exit condition is genuine evidence of product-market fit: proof that a specific, identifiable group of users has found the product valuable enough to return to it (retention), pay for it (revenue), or tell others about it (referral).

MVP 阶段退出标准：MVP 阶段的退出条件是产品与市场契合度的真实证据：证明特定、可识别的用户群体发现该产品的价值足以让他们回来使用（留存）、付费（收入）或告诉他人（推荐）。

MVP stage challenges In the MVP stage, a founder's prime directives are speed and judgment. The challenges here center on whether you can build the right thing, the right way, fast enough to matter, without cutting corners that will cost you later.

MVP 阶段挑战 在 MVP 阶段，创始人的首要指令是速度和判断力。此处的挑战在于：你能否以足够快的速度、用正确的方式构建正确的产品，同时不留下日后会付出代价的捷径。

Agentic technical debt The challenge: Because AI essentially removes every natural bottleneck that once controlled what reaches production, speed is guaranteed. But when speed is the only variable that founders factor into their MVP build, they risk accruing technical debt they'll struggle to pay off.

代理式技术债务 挑战在于：由于 AI 本质上消除了曾经控制产品上线速度的自然瓶颈，速度得以保证。但当创始人仅将速度作为构建 MVP 的考量变量时，他们便面临积累技术债务的风险，日后难以偿还。

Some technical debt is appropriate at the MVP stage, with the understanding that it must be managed before scaling. It builds gradually and can be cleared over time or in a dedicated sprint. AI technical debt, however, compounds.

有些技术债务在 MVP 阶段是合理的，前提是理解必须在规模化之前进行管理。它会逐渐积累，并可在一段时间内或专门的冲刺中清除。然而，AI 技术债务会复利增长。

Without specs and architectural constraints written down somewhere the AI can read, each session re-derives foundational decisions from scratch, and those decisions drift. You end up with a codebase that has no coherent mental model behind it, not because any single piece is bad, but because the pieces were never designed to fit together. That's a real problem, and it does tend to surface late.

如果没有将规范和架构约束写在 AI 可读的地方，每个会话都会从头重新推导基础决策，而这些决策会发生漂移。最终你会得到一个代码库，其背后没有连贯的心智模型，并非因为任何单个部分都很糟糕，而是因为各部分从未被设计为相互契合。这是一个真正的问题，而且往往在后期才显现。

MVP stage

Falling for false product-market fit The challenge: AI tools can generate impressive early numbers, but these are not a guarantee that the market needs your product.

误判产品市场契合度 挑战在于：AI 工具能生成令人印象深刻的早期数据，但这并不保证市场需要你的产品。

Early momentum is one of the most psychologically powerful experiences a founder can have. After weeks or months of validation work and careful, disciplined building, shipping a product feels like confirmation that you were right all along.

早期势头是创始人可能拥有的最具心理影响力的体验之一。经过数周或数月的验证工作和谨慎、自律的构建，发布产品感觉像是确认你一直是对的。

Agentic coding tools can help you reach this moment faster than ever before, but early traction is not the same as product-market fit. Launch energy is generated from ephemeral forces, like your founder's friends, prospective buyers at your investor's other portfolio companies, or a Hacker News headline that drives a spike. Unfortunately, none of these reliably predicts what happens at week six or week twelve when that initial boost has faded.

代理式编码工具可以帮助你比以往更快地达到这一时刻，但早期增长并不等同于产品市场契合。发布能量源于短暂的力量，比如创始人的朋友、投资者其他投资组合公司的潜在买家，或驱动激增的 Hacker News 头条。不幸的是，这些都无法可靠地预测当初初始助推力消退后的第六周或第十二周会发生什么。

Zero-friction scope creep The challenge: When building feels effortless and is nearly free, there's always one more cool feature to add or one more edge case to handle. This scope creep can do more harm than good.

零摩擦范围蔓延 挑战在于：当构建过程感觉毫不费力且几乎免费时，总会想添加一个更酷的功能或处理一个额外的边缘案例。这种范围蔓延可能弊大于利。

Scope creep has always been a startup risk. The difference now is that the traditional forcing function against it—the real cost of engineering time—no longer exists in the same way when adding a feature takes an afternoon instead of a sprint.

范围蔓延一直是初创公司的风险。现在的不同之处在于，传统的对抗机制——工程时间的真实成本——在添加一个功能只需一个下午而非一个冲刺时，不再以同样的方式存在。

The challenge here is that each individual addition is defensible. Of course the product should handle that edge case; of course users will want that workflow .

这里的挑战是，每个单独的添加都是可辩护的。当然，产品应该处理那个边缘情况；当然，用户会想要那个 workflow。

These don't feel like scope creep in the moment because each one takes so little effort to build with agentic coding, but as your product sprawls beyond its original boundaries you risk losing direction and momentum.

这些在当时感觉不像范围蔓延，因为使用代理式编码构建每个功能所需努力如此之小，但当你的产品蔓延超出其原始边界时，你冒着失去方向和动力的风险。

The antidote is a written scope definition created before building begins, describing what the product does, what it deliberately does not do, and the specific evidence from real users that would justify adding something new . This moves the decision point from "should we build this?" to "a critical mass of users have told us they can't get value from the product without this?" Insecure by inexperience The challenge: Founders using AI tools to rush applications to market without first understanding fundamental security principles end up exposing their users to preventable risks.

解药是在开始构建之前创建的书面范围定义，描述产品做什么、它刻意不做什么，以及来自真实用户的具体证据，这些证据将证明添加新事物的合理性。这将决策点从“我们是否应该构建这个？”转变为“大量用户告诉我们，如果没有这个，他们就无法从产品中获得价值？”因经验不足而不安全 挑战：创始人使用 AI 工具在尚未理解基本安全原则的情况下匆忙将应用程序推向市场，最终使其用户暴露于可预防的风险中。

The hard truth is that agentic coding tools generate code that works, not code that is inherently secure. Functional code is easy, because either the feature works or it doesn't. Security vulnerabilities are invisible until they're exploited, which means there's no natural feedback loop to alert a first-time founder that something is wrong. Shipping a live MVP to real users, however, means real data, real exposure, and real consequences if something goes wrong.

硬道理是，代理式编码工具生成的代码能运行，但并非天生安全。功能代码很容易，因为要么功能有效，要么无效。安全漏洞在未被利用前是隐形的，这意味着首次创始人缺乏自然反馈循环来察觉问题。然而，将实时 MVP 推向真实用户，意味着真实数据、真实暴露，以及出错时的真实后果。

Slighting security isn't brand new to AI-native projects. Bootstrapped startups in every era often delayed security considerations until late in the build, sometimes waiting until the verge of production launch. A security review before any user touches your app or solution is the minimum responsible threshold for releasing a minimum viable product into the world.

轻视安全对 AI 原生项目并非新鲜事。每个时代的自筹资金初创公司往往将安全考量推迟到构建后期，有时甚至等到即将投产之际。在任何用户接触你的应用或方案之前进行安全审查，是将最小可行产品推向世界的最低责任门槛。

How Claude can help MVP stage founders Define your architecture before you build Before Claude Code writes a line of production code, use Claude to define and document the architectural decisions that will govern everything built in this stage: the patterns to follow, the dependencies to avoid, the tradeoffs being made and why. This output will serve as a focused architectural context document and establish the guardrails that Claude Code will operate inside.

Claude 如何助力 MVP 阶段创始人 构建前定义架构 在 Claude Code 写入一行生产代码之前，先用 Claude 定义并记录将指导本阶段所有构建工作的架构决策：遵循的模式、避免的依赖、正在做出的权衡及其原因。此输出将作为聚焦的架构上下文文档，并确立 Claude Code 运行的护栏。

Without this context, each session starts from scratch and Claude Code is forced to infer its own structural assumptions. Letting Claude Code build without guardrails produces a codebase that will be functional but structurally incoherent, and iterating on and scaling incoherent codebases is ultimately a waste of time and tokens. Sooner or later there's a point where the code inevitably collapses, forcing you to rebuild from scratch.

若无此上下文，每个会话都从零开始，Claude Code 被迫推断其自身的结构假设。让 Claude Code 在缺乏护栏的情况下构建，会产生功能完备但结构不连贯的代码库，而在结构不连贯的代码库上迭代和扩展，最终是时间和 Token 的浪费。迟早会到达一个节点，代码不可避免地崩溃，迫使你从头重建。

MVP stage

- Exercise: Before opening Claude Code, open Claude and describe what you're building: the core problem it solves, the users it serves, and the scale you realistically expect in the next six months. Ask it to help you define the architectural principles that should govern your MVP build, the dependencies to avoid given your constraints, and the tradeoffs you're consciously accepting at this stage.

- 练习：在打开 Claude Code 之前，先打开 Claude 并描述你正在构建的内容：它解决的核心问题、它服务的用户，以及你在未来六个月内现实预期的规模。请它帮助你定义应指导你 MVP 构建的架构原则、鉴于你的约束应避免的依赖关系，以及你在此阶段有意识接受的权衡。

Next, save this output as `CLAUDE.md` markdown file(s). This is your architectural context document: the first artifact of your build, and the one every subsequent session depends on. `CLAUDE.md` files serve as project-level instructions for Claude Code, providing project-specific context and instructions that are automatically read by the Agent SDK when it runs in a directory. Functionally, they are persistent "memory" for your project.

接下来，将此输出保存为 `CLAUDE.md` Markdown 文件。这是你的架构上下文文档：你构建的第一个工件，也是每个后续会话所依赖的文档。`CLAUDE.md` 文件作为针对项目的指令供 Claude Code 使用，提供项目特定的上下文和指令，当 Agent SDK 在目录中运行时会自动读取这些文件。功能上，它们是你项目的持久“记忆”。

Define and enforce your MVP scope Scope creep without friction is one of the defining failure modes of AI-era MVPs.

定义并执行你的 MVP 范围 零摩擦的范围蔓延是 AI 时代 MVP 的定义性失败模式之一。

Just as you defined and documented your product's application architecture, you also need to define your MVP's scope before a single feature gets built.

正如你定义并记录了产品的应用架构外，你还需要在构建任何功能之前定义 MVP 的范围。

Claude can help you create a scope document that describes what your MVP product does, what it deliberately does not do, and feature amendment criteria:

Claude 可以帮助你创建一个范围文档，描述你的 MVP 产品做什么、它刻意不做什么，以及功能修订标准：

what specific evidence from real users would justify adding something new at this point.

1. 此时，来自真实用户的具体证据是什么，足以证明在此时添加新内容是合理的。

When new feature ideas surface—and they surely will—you use Claude to pressure-test whether it's genuine signal from users or founder enthusiasm dressed up as product thinking.

2. 当新功能点子浮现时——它们肯定会浮现——你利用 Claude 来压力测试：这究竟是来自用户的真实信号，还是被包装成产品思维的创始人热情。

Build your MVP with Claude Code Once architecture and scope are defined, Claude Code becomes the primary MVP build tool. Use it to generate, test, debug, and iterate on your codebase, but treat each session as an execution of product decisions you have already made, not as an opportunity to throw in some new ones.

用 Claude Code 构建你的 MVP 一旦架构和范围确定，Claude Code 就成为主要的 MVP 构建工具。用它来生成、测试、调试和迭代你的代码库，但将每次会话视为对你已做出的产品决策的执行，而非引入新决策的机会。

Start each Claude Code session by (1) revisiting your scope document and (2)

5. 开始每次 Claude Code 会话时，先（1）回顾你的范围文档，然后（2）

providing the model with your CLA UDE.md architectural context document.

6. 向模型提供你的 CLA UDE.md 架构上下文文档。

End each session by updating it with any decisions the session surfaced. The goal is a codebase whose structure you can explain, not just a codebase that runs.

1. 每次会议结束时，用该会议揭示的任何决策更新它。目标是拥有一个结构清晰、你能解释的代码库，而不仅仅是一个能运行的代码库。

- **Exercise:** Create a simple session template for your Claude Code work that includes the architectural context document, the specific task for this session, and any constraints or patterns to observe. At the end of each session, add a brief log entry to the context document that details what was built, what decisions were made, and what assumptions the session introduced. Five minutes of documentation per session is cheap insurance against architectural drift that compounds into an unmanageable codebase.

2. • **练习：**为你的 Claude Code 工作创建一个简单的会议模板，其中包括架构上下文文档、本次会议的具体任务，以及任何需要遵循的约束或模式。在每次会议结束时，向上下文文档添加一条简短的日志条目，详细说明构建了什么、做出了哪些决策，以及该会议引入了哪些假设。每次会议花费五分钟进行文档记录，是防止架构漂移的廉价保险，这种漂移会累积成难以管理的代码库。

Security review before any user touches it As an AI-native startup founder , your responsibility is to know what's in your codebase, understand any potential exposure vectors, and not ship obvious vulnerabilities to real users who are trusting you with their data.

在任何用户接触前进行安全审查 作为 AI 原生初创公司创始人，你的责任是了解代码库中的内容，理解任何潜在的暴露向量，并将明显的漏洞不发送给信任你数据的真实用户。

Claude can do a useful first-pass security review of AI-generated code and can help identify common vulnerabilities. It's a good habit to build into the loop before shipping. It is not a substitute for security tooling, however, or, at higher stakes, a human reviewer—and founders who treat it as one are the ones who end up in the breach stories.

5. Claude 可以对 AI 生成的代码进行有用的首轮安全审查，并帮助识别常见漏洞。在发布前将其纳入流程是一个好习惯。然而，它并非安全工具的替代品，或在 stakes 更高时，也非人类审查员的替代品——那些将其视为替代品而忽视的创始人，最终会出现在漏洞故事里。

Claude Code Security goes further: it scans codebases for security vulnerabilities and suggests targeted patches for human review, surfacing issues that traditional methods can miss.

6. Claude Code Security 更进一步：它扫描代码库中的安全漏洞，并为人类审查建议有针对性的补丁，揭示传统方法可能遗漏的问题。

Note: At the time of this ebook's publication, Claude Code Security is a limited beta release so check current availability before building it into your workflow .

注意：在本电子书出版时，Claude Code Security 仍处于有限测试版阶段，因此在将其纳入工作流程之前，请检查当前的可用性。

- **Exercise:** Before deploying to any real users, run your core application code through Claude with a specific brief: review for authentication and session handling, data exposure in API responses, input validation and injection risks, and dependencies with known vulnerabilities. Treat each finding seriously and assess whether it requires a fix, with human review for anything that touches authentication, secrets, or data handling.

- **练习：**在向任何真实用户部署之前，使用 Claude 运行你的核心应用程序代码，并附带具体指令：审查身份验证与会话处理、API 响应中的数据暴露、输入验证与注入风险，以及具有已知漏洞的依赖项。认真对待每一项发现，并评估其是否需要修复，对于涉及身份验证、密钥或数据处理的内容，需进行人工审查。

MVP stage

Build your measurement framework before launch The founders who mis-identify early traction as product-market fit are typically the same ones who started tracking data after launch, using metrics chosen to assess what was working rather than to surface what wasn't. The antidote is to establish your measurement framework before the first user shows up.

在发布前构建你的衡量框架 将早期增长误判为产品市场契合的创始人，通常是那些在发布后才开始追踪数据的创始人，他们使用的指标是为了评估什么有效，而非揭示什么无效。对策是在第一个用户出现之前建立你的衡量框架。

Use Claude to define which metrics matter for your specific product, what the benchmarks are, and what patterns in the data would constitute genuine product-market fit versus flattering noise. Specifically: set your retention benchmarks, your activation criteria, and your Day 7 and Day 30 targets before releasing your MVP .

使用 Claude 定义哪些指标对你的特定产品至关重要，基准是什么，以及数据中的哪些模式构成真正的产品市场契合度而非奉承的噪音。具体而言：在发布 MVP 之前，设定你的留存基准、激活标准以及第 7 天和第 30 天的目标。

Next, define what a false positive looks like for your specific product: signups without activation, revenue without retention, or initial enthusiasm without repeat usage, for example. When the data arrives, ask Claude to make the adversarial case against your own traction: what would a skeptic say about these numbers?

接下来，定义对你特定产品而言的假阳性表现：例如没有激活的注册、没有留存的收入，或没有重复使用的初始热情。当数据到来时，让 Claude 针对你自己的增长提出反对意见：持怀疑态度的人会如何评价这些数字？

Manage discovery and user feedback logistics Once real users are in the product, the operational layer expands fast. Claude Cowork handles the important-but-tedious work like building and maintaining user contact lists, running outreach sequences, scheduling feedback sessions, triaging bug reports, and tracking iteration cycles. The same MCP integrations that managed discovery logistics in the Idea stage apply here.

管理发现和用户反馈的后勤工作 一旦真实用户进入产品，运营层会迅速扩展。Claude Cowork 处理重要但繁琐的工作，如构建和维护用户联系列表、运行外联序列、安排反馈会议、分类 bug 报告以及追踪迭代周期。在创意阶段管理发现后勤的相同 MCP 集成也适用于此处。

Keep a human in the collection loop for nuanced exploration of user feedback.

在收集循环中保留人类参与，以对用户反馈进行细致入微的探索。

A user saying, for example, "this is great but I wish it could also...," requires interpretation: Is it a core need or a nice-to-have? Is it specific to this customer or representative of a segment? Is the missing feature the real problem, or is something upstream in the onboarding? No tool can answer those questions.

例如，用户说“这很棒，但我希望它也能……”，这就需要解读：这是核心需求还是锦上添花？是针对该客户的特定需求还是代表某个细分群体？缺失的功能是真正的问题，还是用户入职流程上游的某个环节出了问题？没有工具能回答这些问题。

- Exercise: Configure Claude Cowork to run your MVP-stage feedback loop:
 - 练习：配置 Claude Cowork 以运行你 MVP 阶段的反馈循环：
-

draft outreach to your early user list, schedule feedback sessions, design structured intake process for bug reports and feature requests, and write up a weekly synthesis of what's come in. Review the synthesis yourself first; after that, you can ask Claude to analyze the information to catch any significant points you may have overlooked.

起草针对早期用户列表的外联内容，安排反馈会议，设计用于 bug 报告和功能请求的结构化收集流程，并撰写每周汇总报告。先亲自审阅汇总报告；之后，你可以让 Claude 分析信息，以捕捉你可能忽略的任何重要要点。

Iterate toward evidence, not toward completeness The MVP stage ends when you have genuine evidence of product-market fit, no matter how “finished” the product feels. Declaring that you’ve achieved product- market fit and are now ready to move on from the MVP phase to the Launch stage is ultimately a judgement exercise that combines founder intuition with collected evidence. There are some useful litmus tests, though:

向证据而非完整性迭代 当你拥有产品市场契合的真实证据时，MVP 阶段即结束，无论产品感觉多么“完成”。宣布你已实现产品市场契合并准备从 MVP 阶段进入发布阶段，最终是一项结合创始人直觉与收集证据的判断练习。不过，有一些有用的试金石：

- The Sean Ellis test: Ask your active users: "How would you feel if you could no longer use this product?" If more than 40% answer "very disappointed," that's a meaningful PMF indicator.
 - 肖恩·埃利斯测试：询问你的活跃用户：“如果你不再能使用这款产品，你会感觉如何？”如果超过 40% 的人回答“非常失望”，那就是一个有意义的产品与市场契合指标。
-

- The effort test: Pre-product-market fit, retention requires constant intervention, including frequent outreach, incentives, personal follow-up, and a heroic founder energy expended to keep users engaged. Post product- market fit, the product starts doing that work on its own. When things begin pulling instead of pushing, that shift in effort is one of the clearest signals that something real has changed.

- 努力测试：在产品与市场契合之前，留存率需要持续干预，包括频繁的外联、激励、个人跟进，以及创始人投入英雄般的精力来保持用户参与。在产品与市场契合之后，产品开始自行完成这项工作。当事物开始从“推动”变为“拉动”时，这种努力上的转变是某种真实改变发生的最清晰信号之一。

Ultimately , no single data point confirms product-market fit because it's a pattern that has to hold across multiple iteration cycles before you can definitively call it.

最终，没有任何单一数据点能确认产品与市场契合，因为它是一种模式，必须在多个迭代周期中持续成立，你才能确切地称之为产品与市场契合。

Pivot when the evidence demands it What if, even after investing all this work, you just can't seem to arrive at product- market fit? The fact that your results don 't confirm the direction you started from is not failure, it's the system working: the MVP stage is designed to surface this information before you over-invest in the wrong answer.

当证据要求时进行转型 如果即使投入了所有这些工作，你似乎仍无法到达产品市场契合？你的结果未能确认你开始的这一方向，并非失败，而是系统在工作：MVP 阶段旨在你在对错误答案过度投资之前揭示这一信息。

When the data doesn't support your current product, use Claude to work through what that data is telling you.

当数据不支持当前产品时，使用 Claude 来梳理数据所传达的信息。

- Exploring alternative customer segments. Perhaps the users who aren't converting were never the right target to begin with. Often the right audience is already in your data, just underweighted.

- 探索替代客户细分。那些未转化的用户可能从一开始就不是正确的目标。通常，正确的受众已经存在于你的数据中，只是权重不足。

- Adjusting your product's value prop. Maybe you have the correct audience but your MVP is just not resonating with users. An adjustment to onboarding, messaging, or core feature emphasis can potentially fix this without changing what you've built.

- 调整产品的价值主张。也许你拥有正确的受众，但你的 MVP 尚未引起用户共鸣。对入职流程、信息传达或核心功能重点的调整，有可能在不改变已构建内容的前提下解决这一问题。

MVP stage

Stay open to the possibility that the disconnect may run deep enough to require a more fundamental change

保持开放心态，认识到这种脱节可能足够深远，需要更根本的改变。

• Exercise: If you've completed three or more iteration cycles without meaningful movement toward your product-market fit benchmarks, use Claude to run a diagnostic before deciding what to do next. Feed it your retention data, your user feedback, and your original problem hypothesis, and ask it three questions:

• 练习：如果你已完成三次或更多迭代周期，且未朝着产品与市场契合基准取得实质性进展，请在决定下一步行动前使用 Claude 进行诊断。输入你的留存数据、用户反馈和原始问题假设，并向其提出三个问题：

• Is there a segment in this data responding differently than the rest?

• 数据中是否存在一个与其他部分反应不同的细分群体？

• Is the gap between designed value and experienced value a positioning problem or a product problem?

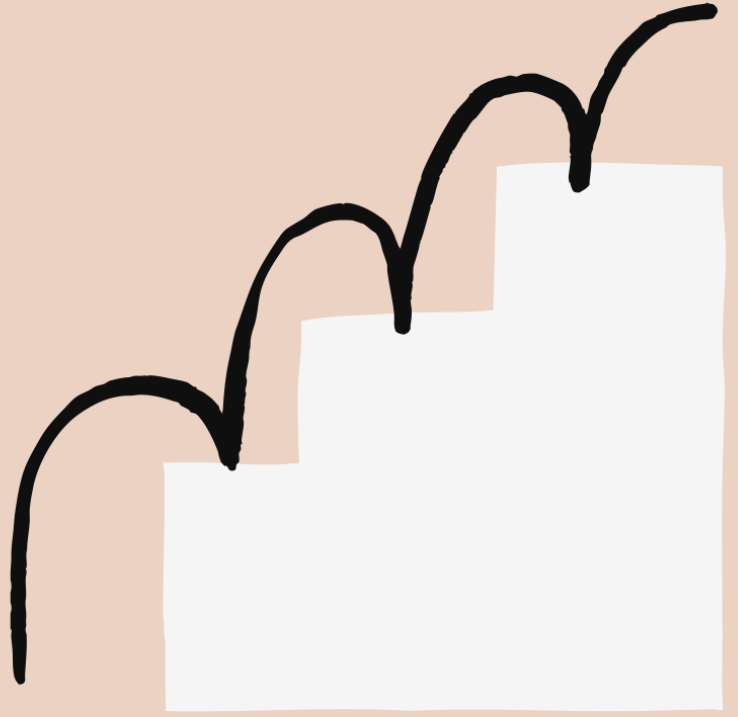
• 设计价值与体验价值之间的差距是定位问题还是产品问题？

• What would have to be true for the current product to find genuine PMF , and is that scenario realistic given what you're seeing?

• 要使当前产品找到真正的 PMF，必须满足什么条件？鉴于你所见，该情景是否现实？

Let the answers determine whether you adjust, pivot, or return to the Idea stage.

• 让答案决定你是调整、转型还是回到创意阶段。



Chapter 5

Launch Stage

21

发布阶段

Launch stage

Chapter 5 Launch stage If the MVP stage was about proving your product deserves to exist, the Launch stage is about proving your business deserves to grow.

第 5 章 发布阶段 如果 MVP 阶段旨在证明你的产品值得存在，那么发布阶段则旨在证明你的业务值得增长。

Launch stage goals In the Launch stage, startup founders must turn early traction into a repeatable, sustainable growth engine. Beyond making your product production-ready, you also must harden the infrastructure underneath it while simultaneously building an actual company around your product.

发布阶段目标 在发布阶段，初创公司创始人必须将早期势头转化为可重复、可持续的增长引擎。除了让你的产品具备量产能力外，你还必须加固其底层基础设施，同时围绕你的产品构建一家真正的公司。

Startups are naturally founder-centric during the Idea and MVP stages because you need the full situational awareness and tight feedback loops. Now, though, founders who still try to personally hold every thread become a Launch stage bottleneck. The goal isn't to remove yourself from the company, but to build operational systems that free your attention for the decisions only a founder can make.

在想法阶段和 MVP 阶段，初创公司天然以创始人中心，因为你需要全面的情境意识和紧密的反馈循环。然而现在，那些仍试图亲自掌控每一根线索的创始人会成为发布阶段的瓶颈。目标并非让你从公司中抽身，而是建立运营系统，释放你的注意力，使其专注于只有创始人才能做出的决策。

Launch stage exit criteria The Launch stage exit condition has three elements:

发布阶段退出标准 发布阶段的退出条件包含三个要素：

1. Growth is repeatable and channel-driven. You're not just retaining users, you're acquiring them predictably through specific channels with understood unit economics: CAC, LTV, and payback period are numbers you know and can defend.

1. Growth is repeatable and channel-driven. You're not just retaining users, you're acquiring them predictably through specific channels with understood unit economics: CAC, LTV, and payback period are numbers you know and can defend. 1. 增长是可重复且由渠道驱动的。你不仅是在留存用户，而是通过特定渠道可预测地获取用户，且单位经济模型清晰：获客成本 (CAC)、生命周期价值 (LTV) 和回本周期是你清楚且能辩护的数值。

2. The product can handle production workloads. Infrastructure is hardened, security and compliance are in order, and reliability holds under real production conditions (not just the conditions you tested for).

2. The product can handle production workloads. Infrastructure is hardened, security and compliance are in order, and reliability holds under real production conditions (not just the conditions you tested for). 2. 产品能够处理生产负载。基础设施已加固，安全与合规已就绪，且在实际生产条件下（而不仅是测试条件）保持可靠性。

3. Operations run without founder bottlenecks. Processes exist and automation is in place. You are no longer the person personally handling support, triage, sprint planning, or reporting.

3. Operations run without founder bottlenecks. Processes exist and automation is in place. You are no longer the person personally handling support, triage, sprint planning, or reporting. 3. 运营不再受创始人瓶颈制约。流程已建立，自动化已到位。你不再亲自处理支持、分诊、冲刺规划或报告。

Launch stage challenges Finding product-market fit is the hardest problem in the early startup lifecycle.

发布阶段挑战 找到产品市场契合度是初创公司生命周期早期最困难的问题。

Now, the founder's challenge becomes keeping it. The Launch stage is where companies that found real product traction may still fall apart if the organization that surrounds and supports the product can't keep up. These are the failure modes to watch for.

现在，创始人的挑战变成了保持它。Launch 阶段是那些找到真实产品牵引力的公司仍可能分崩离析的阶段，如果围绕和支持产品的组织跟不上节奏。这些是需要关注的失败模式。

Technical debt comes due The challenge: The MVP codebase built for speed and validation ran well enough to prove the product worked, but production traffic, new features, and growing complexity are now exposing the shortcuts.

技术债务到期 挑战：为速度和验证而构建的 MVP 代码库运行良好，足以证明产品有效，但现在生产流量、新功能以及日益增长的复杂性正在暴露这些捷径。

At MVP, accumulating some technical debt was a reasonable tradeoff for velocity.

在 MVP 阶段，积累一些技术债务是换取速度的合理权衡。

In the Launch phase, that debt starts accruing interest, and the longer it goes unaddressed, the more expensive it is to fix.

在 Launch 阶段，这笔债务开始产生利息，且搁置越久，修复成本越高。

The solution consists of a systematic architectural audit to identify structural weaknesses, targeted refactoring to address the worst of them, and a meaningful expansion of test coverage so that the next round of feature work doesn't reintroduce the same problems.

解决方案包括：系统性的架构审计以识别结构性弱点，针对性的重构以解决最严重的问题，以及有意义的测试覆盖率扩展，确保下一轮功能工作不再引入相同问题。

Launch stage

The founder becomes the bottleneck The challenge: At MVP , the founder being in every loop was an asset. At Launch, as support volume grows, product decisions stack up, and operational complexity multiplies, that same instinct becomes the constraint.

创始人成为瓶颈 挑战：在MVP阶段，创始人参与每个环节是优势。到了Launch阶段，随着支持量增长、产品决策堆积、运营复杂度倍增，同样的直觉反而成了限制。

The transition from doing the work to designing the systems that do the work is one of the hardest shifts in the startup lifecycle. Because there's rarely a clear moment when it happens, the risk is to miss it entirely and stay in builder mode while the organization stalls around you. Telltale signs that this is happening include decisions that should take an hour now take a week for you to get around to them, support requests that pile up because only you know the answer, and operational tasks that only happen when you personally remember to do them.

从亲自做事到设计做事系统的转变，是初创公司生命周期中最艰难的转型之一。由于很少有一个明确的时刻表明这一转变发生，风险在于完全错过它，而在组织停滞不前时仍停留在构建者模式。迹象包括：本应一小时完成的决策现在需一周才能处理；支持请求堆积，因为只有你知道答案；运营任务仅在你亲自记得时才发生。

The remedy is an all-out audit of everything you're personally handling, from the tiniest task to the most high-stakes decisions, in order to identify what can be systematized, what can be delegated, and what genuinely still merits founder time and attention.

补救措施是对你亲自处理的所有事项进行全面审计，从最微小的任务到最高风险的决策，以识别哪些可以系统化、哪些可以授权、哪些真正仍值得创始人投入时间和关注。

Security and compliance are no longer deferrable The challenge: Keeping security and compliance measures simple was OK for MVP but now, with real users, real data, and potentially enterprise contracts on the table, it becomes a liability.

安全与合规不再可延后 挑战：保持安全与合规措施简单在MVP阶段尚可，但现在面对真实用户、真实数据以及潜在的企业合作合同，它变成了负担。

At MVP , with a handful of beta users and no sensitive data in production, security vulnerabilities were theoretical risks. The hypothetical, however, becomes very real exposure risk the moment your product enters production with real users depending on it. Furthermore, compliance requirements that didn't apply to a prototype definitely apply the moment you're handling customer data, processing payments, or selling into regulated industries.

在 MVP 阶段，只有少量测试用户且生产环境中没有敏感数据，安全漏洞只是理论风险。然而，一旦产品进入生产环境，真实用户开始依赖它，假设就变成了非常现实的风险敞口。此外，原本不适用于原型机的合规要求，一旦你开始处理客户数据、处理付款或向受监管行业销售，就肯定适用了。

The remedy is a systematic security and compliance review before production scale arrives, not after, and treat everything that surfaces as a required remediation—not a suggestion—before the next wave of users arrives.

补救措施是在生产规模到来之前进行系统性的安全和合规审查，而不是之后，并将浮现的所有问题视为必要补救措施——而非建议——在下一波用户到来之前完成。

Expansion before you're ready The challenge: New markets and funding opportunities look like growth opportunities. They can also be where product-market fit goes to die.

准备不足时的扩张 挑战：新市场和融资机会看似增长机遇，但也可能是产品市场契合度消亡之地。

The initial traction you've built is real, but it's also specific to your early audience.

你建立的初始势头是真实的，但它也专门针对你的早期受众。

Expanding too early into a market that's meaningfully different from your original one introduces new user behaviors, compliance requirements, payment infrastructure, and baseline expectations that your product wasn't designed around. Suddenly there are too many new variables and you lose the ability to interpret your own data clearly. You also run the risk of neglecting your original user base while chasing a new and unproven audience.

过早进入与原始市场显著不同的市场，会引入新产品未围绕其设计的新用户行为、合规要求、支付基础设施和基准期望。突然之间，新变量过多，导致你难以清晰地解读自身数据。同时，你在追逐新且未经证实的受众时，还冒着忽视原有用户群的风险。

How Claude can help Launch stage founders All three forms of Claude are in full use in the Launch stage, and they support each other: each tool produces outputs that become inputs for the other two. The results compound organically, and a founder using all three tools together gets more than the sum of their parts.

Claude如何助力Launch阶段创始人 三种形式的Claude在Launch阶段均被充分使用，且相互支持：每个工具的输出都成为另外两个工具的输入。结果有机地复利增长，同时使用三种工具的创始人获得的成果大于各部分之和。

This is what makes the ultra-lean startup model structurally possible. When Claude Code builds the product, Claude Cowork builds the company around it, and Claude helps operationalize this product and organizational knowledge, a small team can run like a company nx its size.

这正是超精益初创公司模式在结构上成为可能的原因。当 Claude Code 构建产品，Claude Cowork 围绕产品构建公司，且 Claude 帮助将产品和组织知识投入运营时，一个小团队就能像一家与其规模相当的公司那样运转。

Remediate technical debt before it compounds Y our MVP codebase works, but it also needs a systematic remediation pass in search of any technical debt that could become a structural liability.

在技术债务复利前进行修复 你的MVP代码库能运行，但也需要一次系统性的修复，以查找可能成为结构性负担的任何技术债务。

First, use Claude Code to run a full architectural audit: identify where the codebase is brittle, any shortcuts that will become expensive to maintain, and where test coverage is thin enough that the next round of feature work will reintroduce the same problems.

首先，使用 Claude Code 运行全面的架构审计：识别代码库中脆弱的地方、任何将成为维护负担的捷径，以及测试覆盖率薄弱的地方，以至于下一轮功能工作将重新引入相同的问题。

Launch stage

Feed Claude Code's audit findings back to Claude to triage and sequence the remediation work: what needs to be fixed before the next release, what can wait a sprint, and what represents acceptable ongoing debt given your current stage.

将 Claude Code 的审计发现反馈给 Claude，以分类和排序修复工作：哪些需要在下次发布前修复，哪些可以推迟一个冲刺周期，以及鉴于您当前的阶段，哪些代表可接受的持续债务。

This is also the moment to document the architectural decisions you made during the MVP stage (the ones that lived in your head because there was no time to write them down). Getting them into a CLAUDE.md now ensures that every future Claude Code session starts from a shared understanding of how the system was designed and why.

这也是记录您在 MVP 阶段做出的架构决策的时刻（那些因为没时间写下来而只存在于您脑海中的决策）。现在将它们写入 CLAUDE.md，可确保每个未来的 Claude Code 会话都对系统如何设计及其原因的共同理解开始。

-
- Exercise: Direct Claude Code to audit your MVP codebase and produce a prioritized list of structural weaknesses, test coverage gaps, and refactoring candidates. Then feed that list to Claude and ask it to sequence the remediation work across your several sprints: any significant issues that you need to address first, things that can be handled in parallel with feature development, and things that can wait.
 - 练习：指示 Claude Code 审计您的 MVP 代码库，并生成一份结构弱点、测试覆盖率缺口和重构候选项的优先级列表。然后将该列表反馈给 Claude，并要求它跨您的多个冲刺周期排序修复工作：任何您需要首先解决的重大问题、可与功能开发并行处理的事项，以及可以推迟的事项。

Build the systems that replace founder attention Building the operational systems that free your attention to handle responsibilities only the founder can tackle requires knowing exactly where your attention is going. Use Claude Cowork to run a structured audit of your current operational load, documenting every recurring task, every decision that lands on your desk, and every workflow that only happens because you personally remember to do it. Then have Claude Cowork categorize this inventory into what can be automated entirely, what needs a human but not necessarily you, and what genuinely requires founder judgment.

构建替代创始人注意力的系统 构建能释放你注意力的运营系统，以便处理只有创始人能承担的责任，需要确切知道你的注意力流向何处。使用 Claude Cowork 对您当前的运营负载进行结构化审计，记录每一项重复任务、每一项落在你桌上的决策，以及每一项仅因你个人记得去做而发生的流程。然后让 Claude Cowork 将这份清单分类为：可完全自动化的部分、需要人类但不一定是你的部分，以及真正需要创始人判断的部分。

Once the audit is complete, use Claude Cowork to design the workflow logic for the automation candidates: what triggers each workflow, what the decision rules are, what the output looks like, and where it goes when it's done.

一旦审计完成，使用 Claude Cowork 为自动化候选项设计工作流逻辑：每个工作流的触发条件是什么、决策规则是什么、输出结果是什么样以及完成后流向何处。

Make security and compliance a product workstream Use Claude Code to surface code-level issues that frequently come up in SOC 2, GDPR, or HIP AA audits and standards that your target market requires. This will surface both vulnerabilities and compliance gaps. Feed those findings to Claude to help you prioritize the remediation work and design the controls, audit logging, and access management that enterprise buyers will ask for before they sign. Note: AI scans are an aid but not a substitute for qualified compliance review.

将安全与合规作为产品工作流，使用 Claude Code 揭示 SOC 2、GDPR 或 HIPAA 审计及目标市场所需标准中频繁出现的代码级问题。这将同时揭示漏洞与合规差距。将这些发现反馈给 Claude，以协助你优先处理补救工作，并设计企业买家签约前会要求的控制措施、审计日志和访问管理。注意：AI 扫描是辅助手段，而非合格合规审查的替代品。

Next, build the compliance workstream into your development cycle rather than running it as a one-time project; compliance documentation needs to be continually maintained and updated. For founders approaching enterprise contracts or international markets, this is also the moment where the Claude Code security scan can help you prepare for an independent security assessment.

接下来，将合规工作流构建到开发周期中，而非作为一次性项目运行；合规文档需要持续维护和更新。对于接近企业合同或国际市场的创始人而言，这也是 Claude Code 安全扫描可帮助您准备独立安全评估的时刻。

- Exercise: Run a code-level security review with Claude Code oriented to the frameworks your target market requires. Feed the output to Claude and ask it to produce two things: a prioritized security remediation sequence and a list of the documentation and controls you'll need to produce to satisfy a compliance review from a prospective enterprise buyer.

- 练习：使用 Claude Code 针对目标市场所需的框架进行代码级安全审查。将输出结果输入 Claude，并要求其生成两样东西：一个优先安全补救序列，以及为满足意向企业买家的合规审查所需生成的文档和控制措施清单。

Stand up the product management processes you've been skipping The Launch stage requires a set of lightweight, repeatable processes that can run without requiring founder intervention to trigger or function. Use Claude to design how your product timeline and work cycles will be structured, what a spec needs to include before Claude Code touches a feature, how bug reports get triaged and routed, and what your weekly metrics report covers and how it's distributed.

建立你一直跳过的产品管理流程。Launch 阶段需要一套轻量、可重复的流程，无需创始人干预即可触发或运行。使用 Claude 设计产品时间线与工作周期的结构、Claude Code 介入功能前规范需包含的内容、Bug 报告如何被分类与路由，以及周度指标报告涵盖的范围与分发方式。

Once process design is done, use Claude Cowork to build and run the operational layer: scheduling sprint ceremonies, routing incoming bug reports to the right place, compiling weekly metrics from your connected data sources, and maintaining the feedback loop that keeps user signals flowing into product decisions.

流程设计完成后，使用 Claude Cowork 构建并运行运营层：安排冲刺仪式，将收到的 Bug 报告路由到正确位置，从你的连接数据源汇总周度指标，并维护将用户信号持续流入产品决策的反馈循环。

• **Exercise:** Ask Claude to design a lightweight product management operating system: a defined sprint cadence, a minimum spec template, a bug triage decision tree, and a weekly metrics brief that pulls from your actual data sources. Then set up Claude Cowork to implement and run the system's recurring operational elements, like scheduling, routing, and report compilation, to happen on schedule without you.

• **练习：**请 Claude 设计一个轻量级产品管理操作系统：定义冲刺节奏、最小规范模板、Bug 分类决策树，以及从你实际数据源提取的周度指标简报。然后设置 Claude Cowork 来实施并运行系统的 recurring 运营要素，如排程、路由和报告汇总，使其在你无需干预的情况下按时发生。



Chapter 6

Scale stage

25

规模化阶段

Scale stage

Chapter 6 Scale stage During the Scale phase, the founder's role re-centers from builder to public-facing executive. The product is still central, but your personal day-to-day work becomes increasingly about the company itself. Your attention must expand to new Scale-stage activities like analyst briefings and IPO roadshows even as you strive to maintain the lean, AI-centered structural advantage.

第 6 章 扩张阶段。在扩张阶段，创始人的角色从构建者重新聚焦为面向公众的高管。产品仍是核心，但你的个人日常工作越来越聚焦于公司本身。你的注意力必须扩展到分析师简报和 IPO 路演等新扩张阶段活动，同时努力保持精益、以 AI 为中心的结构优势。

Scale stage goals The work of scaling technical infrastructure keeps on going, and is now joined by the work of scaling the organization itself and maturing it into a business.

扩张阶段目标。扩展技术基础设施的工作仍在继续，并新增扩展组织本身并将其成熟为业务的工作。

At the scale stage you're looking at going from thousands of users to millions, and from one market to many. At every prior stage, growth was something you could feel your way through by being close to users and adjusting course based on data from tight feedback loops plus a healthy dose of founder instinct. Now, though, the goal is to build systematic growth that's sustained by mature organizational operations.

在规模化阶段，您着眼于从数千名用户扩展到数百万名用户，从单一市场扩展到多个市场。在之前的每个阶段，增长都是您可以通过贴近用户、根据紧密反馈循环的数据以及适量的创始人直觉来调整方向而感受到的。但现在，目标是建立由成熟组织运营所支撑的系统性增长。

For an AI-native startup, your goal should be to build a defensible moat through accumulated depth, stemming from the expertise you've built into your product, your product's depth of integration with the other tools and platforms your users rely on, and the proprietary system data and workflows. The founders who've been building consistently in one direction, on consistent infrastructure, now have something genuinely hard to replicate.

对于一家 AI 原生的初创公司，你的目标应是通过积累的护城河来建立防御优势，这源于你已融入产品的专业知识、产品与你用户所依赖的其他工具和平台的深度整合，以及专有系统数据和流程。那些在一致的基础设施上持续朝一个方向建设的创始人，现在拥有了真正难以复制的东西。

At this stage, public investors, analysts, regulators, enterprise procurement teams, and acquirers apply greater pressure—along with greater skepticism—because the stakes are higher now. Your product and org have to withstand external scrutiny: not just the capabilities of what you've built, but the governance, compliance posture, financial controls, and strategic narrative that surround it.

在此阶段，公众投资者、分析师、监管机构、企业采购团队和收购方会施加更大的压力——伴随着更大的怀疑——因为现在的赌注更高。你的产品和组织必须经受住外部审查：不仅是你所构建的功能，还包括围绕它的治理、合规态势、财务控制和战略叙事。

Scale stage exit criteria The exit condition at Scale is no longer a single milestone but a threshold event:

扩张阶段退出标准。扩张阶段的退出条件不再是单一里程碑，而是一个阈值事件：

the company is sustainable even as the founder is, increasingly, not directly running day-to-day operations. You've demonstrated systematic growth; built organizational governance and compliance infrastructure that satisfies the most demanding external reviewers; and have a solid answer to the question, "If a well-funded incumbent copied your product today, would your users stay?" In practice, this threshold will typically take one of three forms: sustainable profitability at a scale that no longer requires external capital, IPO-readiness, or acquisition. All three require that your growth is systematic and auditable, your product moat stands up under scrutiny, and your organization is operationally mature and sustainable.

即使创始人日益不直接参与日常运营，公司依然可持续。你已经展示了系统性增长；建立了满足最苛刻外部审查者的组织治理和合规基础设施；并对“如果一家资金雄厚的现有竞争对手今天复制了你的产品，你的用户是否会留下？”这一问题有了扎实的回答。在实践中，这一阈值通常表现为三种形式之一：在不再需要外部资本的规模下实现可持续盈利、具备 IPO 准备条件，或被收购。所有这三种情况都要求你的增长是系统化和可审计的，你的产品护城河经得起审查，且你的组织在运营上成熟且可持续。

When this is true, congratulations are in order: your startup has gone from being a bet to being a business.

当这一切成为现实时，值得祝贺：你的初创公司已从一项赌注转变为一项业务。

Scale stage challenges Delegating the operational layer The challenge: Scale-stage operational systems have to run reliably and sustainably without being babysat. For a founder who has been hands-on since day one, that transition can be as much a psychological challenge as a structural one.

Scale stage challenges: 规模化阶段挑战

Delegating the operational layer: 委托运营层

The challenge: 规模化阶段的运营系统必须可靠且可持续地运行，无需专人照看。对于自第一天起就亲力亲为的创始人而言，这一转变既是心理挑战，也是结构挑战。

Scale stage

Your Launch stage work was creating the systems; in the Scale phase, it becomes (1) maturing these systems until they are fully trustworthy and (2) then actually trusting them.

Your Launch stage work was creating the systems; in the Scale phase, it becomes (1) maturing these systems until they are fully trustworthy and (2) then actually trusting them.: 你的启动阶段工作是创建系统；在规模化阶段，它转变为 (1) 完善这些系统直至完全值得信赖，以及 (2) 然后真正信任它们。

This is harder than it sounds. Even if you're a founder who delegates well it's not always obvious what to hand off and what to keep on your plate. Hand off too much, too fast—especially to AI-automated systems—and critical decisions get made without crucial context that only the founder can provide. Hold on too long, though, and you can become a bottleneck.

This is harder than it sounds. Even if you're a founder who delegates well it's not always obvious what to hand off and what to keep on your plate. Hand off too much, too fast—especially to AI-automated systems—and critical decisions get made without crucial context that only the founder can provide. Hold on too long, though, and you can become a bottleneck.: 这比听起来更难。即使你是一位善于授权的创始人，也不总是清楚该移交什么、该保留什么。移交过多或过快——尤其是向 AI 自动化系统移交——关键决策将在缺乏只有创始人能提供的重要背景的情况下做出。但若抓得太久，你又会成为瓶颈。

The fundamental challenge here is identifying the institutional knowledge that lives only in the founder's head or undocumented workflows, and then codifying it into systems that are documented, auditable and transferable.

The fundamental challenge here is identifying the institutional knowledge that lives only in the founder's head or undocumented workflows, and then codifying it into systems that are documented, auditable and transferable.: 这里的根本挑战是识别仅存在于创始人头脑中或未记录工作流程中的机构知识，然后将其编码为已记录、可审计且可转移的系统。

Scaling technical operations The challenge: Customers no longer evaluate only your product; they want to know that your organization can be a dependable infrastructure partner.

Scaling technical operations: 扩展技术运营

The challenge: 客户不再仅评估你的产品；他们希望了解你的组织能否成为可靠的基建合作伙伴。

Technical challenges during the first three startup stages centered on the codebase: building the right solution without accruing technical debt and then hardening security and compliance for real users. Having reached the Scale phase, the challenge now becomes everything built around the codebase;

初创公司前三个阶段的技术挑战集中在代码库：构建正确的解决方案而不积累技术债务，随后为真实用户强化安全与合规。进入规模化阶段后，挑战转变为围绕代码库构建的一切；

creating the support infrastructure, documentation, and reliability guarantees that signal maturity.

creating the support infrastructure, documentation, and reliability guarantees that signal maturity: 创建支持基础设施、文档和可靠性保证，以彰显成熟度。

Larger-scale customers and institutional buyers signing multi-year contracts want these before they'll sign, and they'll also hold you to them once they do.

Larger-scale customers and institutional buyers signing multi-year contracts want these before they'll sign, and they'll also hold you to them once they do: 签署多年合同的大规模客户和机构买家在签约前就希望看到这些，签约后也会据此要求你兑现。

The same AI infrastructure that got you this far, though, helps you build dedicated support functions with defined response times and documentation that a new customer's engineering team can actually use.

不过，助你走到这里的同一套 AI 基础设施，也能帮助你建立专门的职能支持部门，设定明确的响应时间，并提供新客户的工程团队真正能使用的文档。

Scaling organizational functions The challenge: A Scale-stage company generally needs organizational infrastructure like hiring, payroll, accounting, and legal operations, regardless of how many people are running it.

Scaling organizational functions: 扩展组织职能

The challenge: 规模化阶段的公司通常需要组织基础设施，如招聘、薪酬、会计和法律运营，无论由多少人管理。

At Launch, systematizing operations meant automating the workflows consuming founder attention. A Scale-stage startup now needs to grow an even broader, and in some ways more consequential, array of operational functions such as financial reporting, compliance monitoring, contract management, and customer support, to name a few.

在启动阶段，系统化运营意味着自动化那些消耗创始人注意力的工作流程；而处于规模化阶段的初创公司现在需要扩展更广泛、在某些方面更重要的运营职能阵列，例如财务报告、合规监控、合同管理和客户支持等。

Building a GTM function The challenge: Organic growth has a ceiling, and most Scale-stage founders hit it before they've ever had to build a real go-to-market function.

Building a GTM function: 构建市场进入（GTM）职能

The challenge: 有机增长存在上限，大多数规模化阶段的创始人在尚未建立真正的市场进入职能前就已触及该上限。

Idea, MVP , and Launch stage growth often originates from founder-led selling, from a well-timed Product Hunt post to personal relationships with early customers. Organic growth like this works only to a certain point, though, and most startups hit this limit in the Scale phase. Signs include flattening user curves, rising customer acquisition costs, and a pipeline that only moves when the founder is personally involved.

创意、MVP 和发布阶段的增长往往源于创始人主导的销售，从精心时机的 Product Hunt 帖子到与早期客户的个人关系。然而，此类有机增长仅在特定阶段有效，大多数初创公司在成长期会触及这一限制。迹象包括用户曲线趋于平缓、获客成本上升，以及仅当创始人亲自参与时才会推进的管道。

Scale-stage growth requires building a dedicated growth engine to reach new and broader audiences for your product. Most startup founders, though, probably have never had to run things like marketing, sales, and analyst relations programs before. A legit GTM motion requires not just establishing new systems and processes, but also creating a brand voice and story for how you want to talk about your product. Because, at this stage in the startup lifecycle, you're going to need one to reach not only individual new users, but also entire target audiences like investors and enterprise buyers.

成长期增长需要建立专门的增长引擎，以触达产品的新受众和更广泛的受众。不过，大多数初创公司创始人此前可能从未运营过营销、销售和分析师关系等项目。一个真正的 GTM 策略不仅需要建立新系统和流程，还需要为产品打造品牌声音和故事。因为，在初创公司生命周期的这一阶段，你需要它来触达不仅是个体的新用户，还包括投资者和企业买家等整个目标受众。

Fortunately , the GTM function doesn't have to be large to be effective, and the same AI infrastructure that built the product can bootstrap bringing it to market.

幸运的是，GTM 职能不必庞大才能有效，构建产品的同一 AI 基础设施即可启动将其推向市场的过程。

How Claude can help Scale stage founders Early startup stages use Claude as foundational infrastructure for the product itself: a research partner for validating the idea, the engineering team that designs and builds the prototype, and the AI operational layer that makes a single-founder startup possible. AI-native startup founders who reach the Scale

Claude 如何帮助处于扩张阶段的创始人：早期初创阶段将 Claude 用作产品本身的基础设施：作为验证想法的研究伙伴、设计并构建原型的工程团队，以及使单人创始人初创公司成为可能的 AI 运营层。达到扩张阶段的 AI 原生初创公司创始人

Scale stage

stage can now use Claude, Claude Code, and Claude Cowork to keep scaling the same way they built.

将日常任务移交给 Claude Cowork

Handing off day-to-day tasks to Claude Cowork Start the Scale stage with a clear-eyed view of where you most need to invest your time and attention now , which can be a challenge for first time founders who've never built a business before. Claude can help by building the list of things only you should be doing at this stage, which could include things like product narrative decisions, board relationships, enterprise deals, and founder- to-founder conversations. Anything not on that list is a candidate for delegation or Claude Cowork automation.

将日常任务移交给 Claude Cowork：以清醒的视角开启规模化阶段，明确此刻你最需要投入时间和精力 的地方，这对从未建立过企业的初次创始人来说可能是一个挑战。Claude 可以通过列出本阶段只有你才应亲自处理的事项来提供帮助，这些事项可能包括产品叙事决策、董事会关系、企业级交易以及创始人之间的对话。任何不在该清单上的事项，都是委托或 Claude Cowork 自动化的候选对象。

- Exercise: Use Claude to produce a bottleneck map of your current operational layer: every workflow , decision, and approval currently routed through you.

[结论] 练习：利用 Claude 生成当前运营层的瓶颈地图：梳理所有目前经由你流转的工作流程、决策与审批环节。

Now , ask Claude to extrapolate what happens to each one when you're unavailable for a week. The workflows that stall are the ones where you are still hands-on enough to derail progress.

[结论] 随后，请 Claude 推演当你缺席一周时，上述每一项会发生什么。那些陷入停滞的工作流程，正是你仍亲力亲为到足以阻碍进展的环节。

How do these map to the inventory of founder priorities and responsibilities you made with Claude ?

这些如何对应于你与 Claude 共同制定的创始人优先事项和责任清单？

Next, it' s time to pressure-test that the systems you've already built are actually ready to scale with your business as it grows.

接下来，是时候压力测试你已构建的系统，确保它们确实能随业务增长而扩展。

- Exercise: Use Claude to map your current workflows, and then ask it what happens to each one when you're unavailable for a week. The workflows that stall are the ones where handoff criteria, escalation paths, or exception handling still need tightening. Claude can help analyze the failure points and recommend appropriate fixes so you can update or replace Claude Cowork automations as necessary.

- 练习：使用 Claude 映射你当前的工作流程，然后询问当你在场一周时每个流程会发生什么。停滞的流程是那些交接标准、升级路径或异常处理仍需完善的流程。Claude 可帮助分析故障点并推荐适当修复方案，以便你按需更新或替换 Claude 协作自动化。

Scale technical operations into enterprise-grade infrastructure As you scale, buyers need reassurance that your product and your organization can be trusted as long-term infrastructure. Technical work still goes on inside the codebase as always, but now there is technical work around the codebase to handle, too.

将技术运营扩展至企业级基础设施：随着规模化推进，买家需要确信你的产品和组织能够作为长期基础设施值得信赖。代码库内部的技术工作一如既往地继续进行，但现在也需要处理围绕代码库的技术工作。

The first step is to convert institutional knowledge into a system that scales.

第一步是将机构知识转化为可扩展的系统。

Use Claude to draft and maintain the written infrastructure that enterprise procurement expects to see, including product documentation, support playbooks, and SLAs.

使用 Claude 起草并维护企业采购方期望看到的书面基础设施，包括产品文档、支持手册和 SLA。

In parallel, direct Claude Code to audit and harden the codebase against the specific reliability and security standards that enterprise contracts require, and to build out the technical support infrastructure that Discord-based community support never had to provide: logging, monitoring, incident response tooling, and the observability layer that makes SLAs actually enforceable.

同时，指挥 Claude Code 针对企业合同要求的具体可靠性和安全标准，审计并加固代码库，并构建基于 Discord 的社区支持从未需要提供的技术支持基础设施：日志记录、监控、事件响应工具以及使 SLA 真正可执行的观测层。

Claude Cowork then runs the operational layer of enterprise support itself: ticket routing, escalation workflows, documentation updates triggered by product changes, renewal tracking, and the reporting cadences that enterprise customer success relies on. Together, these three give a small team the support posture of a much larger organization, which is exactly what signing a multi-year enterprise contract requires you to demonstrate.

Claude Cowork 随后运行企业支持本身的运营层：工单路由、升级工作流、由产品变更触发的文档更新、续订跟踪，以及企业客户成功所依赖的报告节奏。这三者共同使小团队具备大型组织的支持态势，而这正是签署多年期企业合同所要求展示的内容。

- Exercise: Pick your three most demanding prospects or identify three ideal customers for your product that you'd love to sign. Ask Claude to produce a gap analysis: what documentation, SLAs, and support infrastructure would an enterprise procurement team at each of these accounts expect to see before signing a multi-year contract, and where do you currently fall short? Use the output to sequence the technical and documentation work across Claude Code and Claude Cowork.

- 练习：选择您三个需求最苛刻的潜在客户，或确定三个您渴望签约的理想客户。请 Claude 生成差距分析：这些客户的企业采购团队在签署多年期合同前，会期望看到哪些文档、SLA 和支持基础设施，而您目前在哪里存在不足？利用输出结果，在 Claude Code 和 Claude Cowork 之间安排技术和文档工作的顺序。

Build a real GTM function Founder hustle got you this far, but scaling your startup requires creating and implementing an actual go-to-market strategy. AI can help you build, then and run, that complete GTM engine.

构建真正的 GTM 职能：创始人的拼搏让你走到这一步，但规模化你的初创公司需要创建并实施真正的市场进入策略。AI 可以帮助你构建并运行这个完整的 GTM 引擎。

Claude can assist with building foundational GTM resources from scratch:

Claude 可以协助从零开始构建基础的市场进入（GTM）资源：

market segmentation, messaging architecture, analyst relations strategy, sales playbooks, and the investor-facing metrics narratives that matter once you're talking to public investors, enterprise buyers, and Wall Street analysts. Each of these audiences has its own vocabulary and evaluates you against its own

市场细分、信息架构、分析师关系策略、销售剧本，以及当你与公众投资者、企业买家和华尔街分析师交谈时至关重要的面向投资者的指标叙事。每个受众群体都有自己的词汇，并根据其自身的标准评估你。

Scale stage

standards; Claude's job is to translate your product's value props into a product marketing approach that's relevant for each audience segment.

标准来评估你；Claude 的工作是将你的产品价值主张转化为与每个受众细分相关的产品营销方法。

Now , Claude Cowork can become your tactical execution layer: content pipelines, outbound sequences, analyst briefing logistics, newsroom and PR cadences, CRM hygiene, pipeline reporting, and the many recurring cycles that turn GTM strategy into actual commercial motion.

现在，Claude Cowork 可以成为你的战术执行层：内容管道、外联序列、分析师简报后勤、新闻室和公关节奏、CRM 卫生、管道报告，以及将 GTM 战略转化为实际商业运动的许多循环周期。

Where the GTM motion requires product marketing infrastructure—interactive demo environments, integration documentation, sandbox tenants, API references, technical one-pagers—Claude Code can build it for you. Buyers expect to evaluate your product technically and, in the Scale phase, a Loom video and a sales deck no longer suffice. This is also the infrastructure that lets your GTM motion run asynchronously: a well-built demo environment closes deals while you're in board meetings.

当 GTM 运动需要产品营销基础设施时——交互式演示环境、集成文档、沙盒租户、API 参考、技术一页纸——Claude Code 可以为你构建。买家期望从技术层面评估你的产品，而在规模化阶段，一段 Loom 视频和一份销售演示文稿已不再足够。这也是让 GTM 运动异步运行的基础设施：一个构建良好的演示环境可以在你参加董事会会议时促成交易。

Turning domain expertise and institutional knowledge into AI context Many ultra-lean startup founders are building highly specific apps or tools for a real-world problem they experience or observe first-hand in a particular sector .

将领域专业知识和机构知识转化为 AI 上下文：许多超精简初创公司的创始人正在为他们在特定行业中亲身经历或观察到的现实问题，构建高度专用的应用程序或工具。

Agentic AI now makes it possible for founders who have never written a line of code to use their domain expertise to build products that solve sophisticated problems. Claude, Claude Code, and Claude Cowork each play a part in converting founder knowledge into compounding product specificity .

代理式 AI 现在使得从未写过一行代码的创始人能够利用其领域专业知识，构建解决复杂问题的产品。Claude、Claude Code 和 Claude Cowork 各自在将创始人知识转化为累积的产品特异性方面发挥作用。

Using Claude to capture, organize, and refine founder knowledge puts domain expertise somewhere the product can reach. Through extended conversations, projects, and memory, a founder can share everything they know—industry jargon, regulatory gotchas, edge cases, frustrations, reasons why the obvious answers to this problem don't work—into a structured, searchable context. Skills can then codify recurring workflows (e.g., "how I audit a commercial lease," "how I triage a patient intake form") into reusable routines Claude runs the same way every time. Over months, this becomes a proprietary knowledge substrate that no generalist AI can match.

使用 Claude 捕获、组织和提炼创始人知识，使领域专业知识处于产品可触及的位置。通过持续的对话、项目和记忆，创始人可以分享他们所知道的一切——行业术语、监管陷阱、边缘案例、挫折、以及为什么该问题的显而易见的答案行不通——并将其转化为结构化的、可搜索的上下文。技能随后可将重复的工作流（例如，“我如何审计商业租约”、“我如何分诊患者登记表”）编码为 Claude 每次以相同方式运行的可重用例程。数月之后，这将成为通用 AI 无法匹敌的专有知识基础。

Externalizing your domain knowledge with Claude becomes invaluable for encoding industry-specific edge cases into your product: a generalist AI medical billing tool breaks on 340B drug program claims, for example, but yours has specific logic for them. Claude Code helps you translate common frustrations experienced by other professionals in your field into validation logic, prompt refinements, or an MCP integration with a niche industry system your competitors haven't heard of. As a result, your app or tool's depth and breadth both continually compound in a way that competitors simply can't replicate.

将你的领域知识与 Claude 外部化，对于将行业特定的边缘案例编码进你的产品中变得极具价值：例如，通用 AI 医疗账单工具会在 340B 药物计划索赔上出错，而你的工具则拥有针对它们的特定逻辑。Claude Code 帮助你把你所在领域其他专业人士经历的常见挫折转化为验证逻辑、提示优化，或与竞争对手尚未听闻的利基行业系统的 MCP 集成。因此，你的应用或工具的深度和广度以竞争对手无法复制的方式持续复利增长。

• **Exercise:** Identify one edge case a generic competitor would definitely get wrong in your vertical. Work with Claude Code to build a dedicated test case for it (not a unit test) based on a scenario you've actually seen. Every time a similar edge case surfaces, add it. Your test suite becomes a map of your moat.

• **练习：**识别一个通用竞争对手在你的垂直领域肯定会出错的边缘案例。与 Claude Code 合作，基于你实际见过的场景为其构建一个专用测试用例（而非单元测试）。每当出现类似的边缘案例，就将其添加进去。你的测试套件将成为你护城河的地图。

Compound accumulated user data into a defensible advantage As users interact with your product, they generate behavioral signals (i.e., which outputs they accept and which they reject), which informs the product roadmap.

将累积的用户数据复利化为可防御的优势 当用户与你的产品互动时，他们会产生行为信号（即他们接受哪些输出和拒绝哪些输出），这为产品路线图提供信息。

Over time, you'll learn the specific patterns, preferences, and edge cases of your particular user base. This is what we mean by compounding value: each improvement makes the product more useful, which drives more usage, which creates more feedback, which drives more improvement.

随着时间的推移，你将了解特定用户群的具体模式、偏好和边缘案例。这就是我们所说的复利价值：每次改进都使产品更有用，从而驱动更多使用，进而产生更多反馈，再驱动更多改进。

This data is time-locked, context-specific, and impossible for a copycat to recreate: you simply can't buy the behavioral fingerprint of thousands of users who've been refining their workflows inside your product.

这些数据是时间锁定的、上下文特定的，且模仿者无法重现：你 simply 无法买到数千名用户在你的产品内不断优化工作流的行为指纹。

Claude can help audit whatever user interaction data you've collected, identify the highest-signal behavioral patterns within it, and design the feedback loop that turns ongoing usage into systematic model improvement.

Claude 可以帮助审计您收集的任何用户交互数据，识别其中信号最高的行为模式，并设计反馈循环，将持续使用转化为系统的模型改进。

- Exercise: Feed Claude a summary of your product's interaction data: what you've been collecting, how long you've been collecting it, and what you know about how users engage with your product over time. Ask it to identify the three highest-signal behavioral patterns in that data and design a feedback loop that turns each one into a systematic model improvement. Then ask it to help you draft a one-page moat narrative to inform product marketing: the story of how your data flywheel works, how long it's been spinning, and why a well- resourced competitor starting today couldn't replicate it in under two years.

- 练习：向 Claude 提供您产品交互数据的摘要：您收集了什么、收集了多久，以及您对用户随时间如何与产品互动的了解。要求它识别该数据中信号最高的三个行为模式，并设计一个反馈循环，将每个模式转化为系统的模型改进。然后要求它协助您起草一页护城河叙事，以指导产品营销：讲述您的数据飞轮如何运作、已运转多久，以及为何一个资源充足的竞争对手从今天开始无法在两年内复制它。

Scale stage

Create workflow lock-in Compounding data network effects make your product harder to replicate, but user workflow lock-in makes your product harder to leave. The longer users run your product inside their daily operations, the more deeply it gets embedded in how they actually work. They've built automations on top of it, trained people to use it, and connected it to their data sources and other tools. The prompts they've developed, the workflows they've refined, and the outputs they've standardized have all been shaped around what your product does and how it does it. At this point, switching goes from product decision to full scale operational project.

创建工作流锁定 复利数据网络效应使你的产品更难复制，但用户工作流锁定使你的产品更难离开。用户在你的日常运营中使用你的产品时间越长，它就越深地嵌入到他们实际工作方式中。他们在其上构建了自动化，培训人员使用它，并将其连接到他们的数据源和其他工具。他们开发的提示、优化的工作流和标准化的输出都已围绕你的产品做什么以及怎么做而塑造。此时，切换从产品决策变为全面规模的运营项目。

The first step in creating workflow lock-in is asking Claude to map your current customer base by integration depth. For each customer segment, identify what workflows they've built on top of your product and which integrations they depend on. This shows where your product is sticking, and where it needs to go deeper.

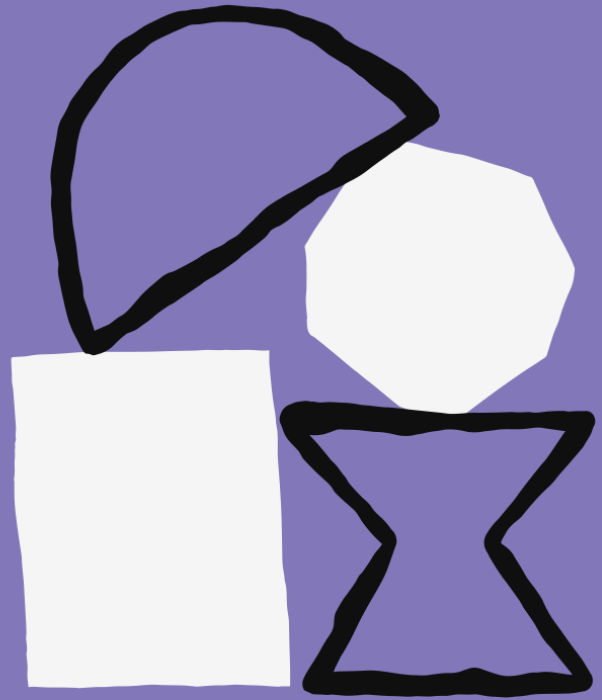
创建工作流锁定的第一步是要求 Claude 按集成深度映射您当前的客户群。对于每个客户细分，识别他们在您的产品之上构建了哪些工作流，以及他们依赖哪些集成。这显示了您的产品在哪里稳固，以及需要在哪里更深入。

The more integrations you offer, the more surface area a customer has to construct workflows that rely on your product. Claude Code helps you quickly spin up native integrations with the data pipelines, project management tools, and other systems that your target users depend on. Claude Code can also build the APIs, webhooks, and SDKs that let customers not just use your product, but build on top of it—the deepest form of lock-in

您提供的集成越多，客户构建依赖您产品的工作流的表面积就越大。Claude Code 帮助您快速启动与数据管道、项目管理工具和其他目标用户依赖的系统之间的原生集成。Claude Code 还可以构建 API、webhooks 和 SDK，让客户不仅能使用您的产品，还能在其之上构建——这是最深层次的锁定

- Exercise: Ask Claude to help you build a workflow integration audit for your top ten customers. For each one, document the automations they've built, the integrations they depend on, the team workflows that run through your product, and your estimate of their switching cost. Then ask Claude to identify the patterns across the group: what types of integration create the deepest lock -in for your specific product, and what you could build or enable to deepen integration for customers who are currently at the surface.

- 练习：请 Claude 协助你为你的前十大客户构建工作流集成审计。为每位客户，记录其已构建的自动化流程、所依赖的集成、流经你产品的团队工作流，以及对其转换成本的估算。随后请 Claude 识别该群体中的模式：哪些类型的集成对你的特定产品创造了最深的锁定效应，以及你可以构建或启用什么来加深目前仅处于表面层的客户的集成。



Chapter 7

Same job, new rules

31

同样的工作，新的规则

Same job, new rules

Chapter 7 Same job, new rules In the AI area, the founder's job hasn't changed: find a real problem, build something that solves it, and scale it into a company that matters. What's changed is the path to get there. Across the four stages—Idea, MVP, Launch, and Scale—AI compresses quarters into weeks.

第 7 章 同样的工作，新的规则 在 AI 领域，创始人的工作没有改变：找到一个真正的问题，构建解决它的事物，并将其扩展为一家重要的公司。改变的是通往那里的路径。在四个阶段——概念、MVP、发布和扩展——AI 将季度压缩为周。

Validation cycles that used to take months now take afternoons. A working prototype no longer requires a co-founder with the right stack; it requires a clear problem and a few focused sessions with a coding agent. Launch readiness compresses from a pre-launch scramble into a continuous workstream. And at scale, the operational weight that used to force early hires into firefighting roles can increasingly be handed off to AI, freeing your team to spend their attention on the judgment calls that become your moat.

验证周期从过去的数月缩短为只需几个下午。一个可工作的原型不再需要拥有正确技术栈的联合创始人；它只需要一个清晰的问题和几次与编码智能体的专注会话。上市准备从发布前的忙乱压缩为持续的工作流。而在规模化阶段，过去迫使早期员工陷入救火角色的运营重担，现在可以越来越多地移交给人工智能，让你的团队将注意力集中在那些成为你护城河的判断性决策上。

The bottlenecks are no longer what you can build, but what you choose to build.

瓶颈不再是你能构建什么，而是你选择构建什么。



Resources

Resources

Resources Building with Claude

资源 使用 Claude 构建

- Building AI Agents for Startups: Shares how startups use agents to become less founder-dependent as they scale.

- 为初创公司构建 AI 智能体：分享初创公司如何利用智能体在规模化过程中降低对创始人的依赖。

- Claude Code docs: Carries builders from initial installation to advanced agentic workflows. Pro-tip: get started with the "How Claude Code works" overview .

- Claude Code 文档：引导构建者从初始安装到高级代理工作流。提示：从“Claude Code 工作原理”概述开始。

- Claude Code best practices: Covers patterns that have worked inside Anthropic and across engineering teams — context management, permissions, planning, and verification workflows.

- Claude Code 最佳实践：涵盖 Anthropic 内部及工程团队中行之有效的模式——上下文管理、权限、规划和验证工作流。

- Using CLAUDE.md files: Walks through how to configure Claude Code for your specific codebase. Essential reading for MVP-stage founders setting up their development environment.

- 使用 CLAUDE.md 文件：详解如何为特定代码库配置 Claude Code。MVP 阶段创始人搭建开发环境时的必读内容。

- Claude Code power user tips: Highlights workflow patterns from the Claude Code team itself , including parallel sessions and verification loops.

- Claude Code 高级用户技巧：突出 Claude Code 团队自身的工作流模式，包括并行会话和验证循环。

- Get started with Claude Cowork: Shares how teams can set up Claude Cowork and start implementing skills, plugins, and other features that scale its impact across your startup.

- 开始使用 Claude Cowork：分享团队如何设置 Claude Cowork，并开始实施技能、插件及其他功能，以扩大其在初创公司中的影响。

- Tutorials: claude.com/resources/tutorials offers a searchable list of hands-on walkthroughs for specific tasks.

- 教程：claude.com/resources/tutorials 提供可搜索的手把手任务指南列表。

Founder stories

创始人故事

- How three YC startups built their companies with Claude Code: Examining how HumanLayer (F24), Ambral (W25), and Vulcan Technologies (S25) used Claude to get prototypes to market fast and scale AI-powered platforms with agentic coding workflows.

- 三家 YC 初创公司如何利用 Claude Code 建立其公司：考察 HumanLayer (F24)、Ambral (W25) 和 Vulcan Technologies (S25) 如何使用 Claude 快速将原型推向市场，并通过代理编码 workflow 扩展 AI 驱动的平台。

- GC AI’s founders used domain expertise to build a responsive, Claude-powered legal platform for how in-house teams actually work: company-specific playbooks, cross-functional stakeholders, and variable risk tolerance thresholds

- GC AI 的创始人利用领域专业知识，为内部团队的实际工作方式构建了一个响应迅速、由 Claude 驱动的法律平台：包括公司特定的作战手册、跨职能利益相关者以及可变的风险容忍度阈值。

- Carta Healthcare uses Claude to power their clinical abstraction platform, processing 22,000 surgical cases per year and reducing data abstraction time by 66%.

- Carta Healthcare 利用 Claude 驱动其临床抽象平台，每年处理 22,000 例手术案例，并将数据抽象时间减少了 66%。

- Anything, powered by Claude and the Agent SDK, has helped 1.5 million users turn ideas into working software products without writing code, including a non-technical founder who built and is already selling a full recruiting platform. Anything’s AI agent handles the full build so solopreneurs can double down on their domain expertise.

- Anything 由 Claude 和 Agent SDK 驱动，已帮助 150 万用户无需编写代码就将创意转化为可运行的软件产品，其中包括一位非技术创始人，他构建并正在销售一个完整的招聘平台。Anything 的 AI 代理处理整个构建过程，使单人创业者能加倍专注于其领域专业知识。

- Cogent is an applied AI lab building agents to automate critical enterprise security tasks. The startup uses Claude as the reasoning layer for agents that automate investigation, prioritization, and remediation across the full vulnerability lifecycle.

- Cogent 是一家应用 AI 实验室，致力于构建代理以自动化关键的企业安全任务。该初创公司使用 Claude 作为推理层，以自动化整个漏洞生命周期中的调查、优先级排序和修复工作。

- Airtree uses Claude Cowork as the center of its operations infrastructure, uniting data that used to be scattered across a dozen different tools and teams.

- Airtree 使用 Claude Cowork 作为其运营基础设施的核心，将原本分散在十几种不同工具和团队中的数据统一起来。

Now , when one person builds a workflow automation with skills, everyone in the organization can use it to do all the things on their to-do list that never got done.

现在，当一个人利用技能构建工作流自动化时，组织中的每个人都可以使用它来完成待办事项列表中那些从未完成的任务。

- Duvo builds AI agents that run procurement, supply chain, and category management processes across ERPs, supplier portals, spreadsheets, email, and even phone calls. Duvo is built entirely on Claude, using the Agent SDK to orchestrate across workflows.

- Duvo 构建 AI 代理，在 ERP、供应商门户、电子表格、电子邮件甚至电话中运行采购、供应链和品类管理流程。Duvo 完全基于 Claude 构建，使用 Agent SDK 协调跨工作流。

- Zingage is an AI agent platform built for 24/7 automated operations for home- care agencies. The startup uses Claude’s structured tool calling to orchestrate across an EMR and multiple communication channels, and Claude’s contextual reasoning to build agents that can give nuanced, patient-tailored outcomes rather than pattern-matching to the most common response.

- Zingage 是一个专为居家护理机构打造的 24/7 自动化运营 AI 代理平台。该初创公司利用 Claude 的结构化工具调用在电子病历 (EMR) 和多个通信渠道之间进行协调，并利用 Claude 的上下文推理能力构建代理，以提供细致、针对患者的结果，而非仅对最常见响应进行模式匹配。

- Kindora is an AI-powered platform built by a nonprofit executive who used Claude Sonnet to build a desperately-needed tool for intelligently matching charities with funders. After filtering thousands of matches down to the

- Kindora 是一个由非营利组织高管构建的 AI 驱动平台，该高管使用 Claude Sonnet 构建了一个急需的工具，用于智能地将慈善机构与资助方进行匹配。在将数千个匹配项筛选为少数值得追求的选项后，

Resources

few worth pursuing, Kindora's MCP connector lets nonprofits access its prospecting tools directly within Claude.

Kindora 的 MCP 连接器让非营利组织能够直接在 Claude 内部访问其潜在客户挖掘工具。

- Wordsmith was founded by a lawyer-turned-CTO to provide reliable AI- powered legal technology for in-house legal teams. Claude is the reasoning engine for Wordsmith's contract review, agreement drafting, and document review capabilities, and the startup's engineering team uses Claude Code for building and evolving the platform itself.

- Wordsmith 由一位从律师转型为 CTO 的人创立，旨在为内部法务团队提供可靠的 AI 驱动的法律技术。Claude 是 Wordsmith 合同审查、协议起草和文档审查能力的推理引擎，该初创公司的工程团队使用 Claude Code 来构建和演进平台本身。

Startup support and opportunities

初创公司支持与机会

- Anthropic Startups Program: For startups working with Anthropic's V C partners, the program provides free API credits, the highest tier of publicly available rate limits, and invitations to exclusive founder events and workshops.

- Anthropic 初创公司计划：针对与 Anthropic 的 VC 合作伙伴合作的初创公司，该计划提供免费的 API 额度、公开可用的最高层级速率限制，以及独家创始人活动和研讨会的邀请。

- Claude community: Forums and community spaces for builders.

- Claude 社区：面向构建者的论坛和社区空间。

- Live learning resources: Conferences, webinars, livestreams, and recordings.

- 实时学习资源：会议、网络研讨会、直播和录像。

Resources

claude.ai

claude.ai

本中英双语学习版基于原英文材料整理制作，仅用于学习、阅读与交流。

整理制作：公众号「AI进化俱乐部」